



Departamentul Automatică și Informatică Industrială
Facultatea Automatică și Calculatoare
Universitatea Națională de Știință și Tehnologie
POLITEHNICA din București



LUCRARE DE DIPLOMĂ

Sistem pentru detecția și raportarea evenimentelor de securitate în rețele industriale de comunicații

Conducător științific: Conf. dr. Ing. Florin Daniel Anton

Asistent coordonator: Drd. Ing. Ana-Theodora Mitran

Absolvent: Theodor Predescu

2026

Abstract

Lucrarea prezentată introduce o aplicație de securitate pentru sistemele industriale de control. Soluția cuprinde două componente ce sunt destinate calculatoarelor industriale, o componentă ce reprezintă serverul sistemului și o aplicație desktop de control general. Se utilizează reguli de acces pe bază de politici bine definite și controlate de administratorul sistemului. Prin natura modulară și funcționalitatea de a acumula informațiile inspectate, are flexibilitatea de a include ulterior alte tipuri de inspecții a traficului, precum detecții pe baza anomaliilor sau a semnăturilor. Aplicația se concentrează pe configurarea și inspecția de la distanță a setărilor de filtrare, monitorizarea intruziunilor petrecute la nivelul componentelor de interes și vizualizarea grafică a comunicărilor din rețeaua locală. Deși complexitatea filtrării și fiabilitatea aplicației nu reprezintă un factor de atractivitate pentru sistemele industriale mari, ele afirmă că aceste concepte prezentate și componente logice pot participa la dezvoltarea unei soluții complexe, ce facilitează toate nevoile moderne de securitate și protecție împotriva intruziunilor.

Cuvinte cheie: sisteme industriale de control, securitate, detecție a intruziunilor, acces pe bază de politici, monitorizare, filtrare

The presented thesis lays the foundation for a security model designed for industrial control systems. This proposed solution consists of two components intended to run on industrial equipment, a component that represents the server of the system and a desktop application for general control. The system uses access control rules based on well-defined policies configured and managed by the system administrator. Through its modular nature and its ability to collect inspection information, the solution provides the necessary flexibility to integrate additional traffic inspection methods in the future, such as anomaly-based and signature-based detection methods. The application focuses on the remote configuration and inspection ability of the filtering settings, monitoring intrusions events occurring at the level of the protected components, and providing graphical visualization of communication members within the local network. Although the filtering capabilities and the overall reliability of the proposed implementation may not be sufficient for large-scale industrial environments, the concepts and logical components presented in this work can contribute to the development of a more comprehensive solution, capable of addressing modern security requirements and protection against intrusions.

Keywords: Industrial Control Systems, cybersecurity, intrusion detection, policy-based access control, monitoring, traffic filtering.

Cuprins:

Introducere	4
Prezentarea domeniului.....	5
Sisteme industriale de control	5
Metode de comunicare utilizate	6
Expunerea sistemelor ICS	10
Tipuri de atacuri în sisteme ICS	11
Descrierea problemei și soluția propusă	14
Soluții existente de securitate a sistemelor ICS	15
Prezentarea aplicației.....	19
Detalii de implementare	21
Componenta de intermediere	24
Componenta centrală	28
Componenta de administrare.....	32
Fluxul de funcționare al sistemului.....	37
Experiența utilizatorului.....	40
Concluzii și planuri de viitor	47
Bibliografie.....	49

Introducere

Odată cu avansarea tehnologică din ultimele decenii și apariția unor necesități noi în domeniul tehnologic, procesele de automatizare și digitalizare au suferit creșteri semnificative în aria conectivității sistemelor industriale. Infrastructuri care în trecut funcționau în medii izolate se doresc a fi integrate în rețele locale complexe, iar în multe din aceste cazuri se caută soluții ce facilitează conectarea indirectă la Internet. Metodele ce realizează și securizează această nouă viziune se bazează pe monitorizarea integrală și administrarea centralizată a proceselor industriale, însă, cu toate acestea, ele nu reprezintă soluții imune la atacuri.

Sistemele industriale de control (*Industrial Control Systems – ICS*) reprezintă componente esențiale ale infrastructurilor moderne de control și monitorizare a proceselor industriale axate pe asigurarea siguranței operaționale și a bunei desfășurări a proceselor. Funcționarea corectă a acestor sisteme este, prin urmare, de mare importanță, iar compromiterea lor poate duce atât la pierderi economice semnificative, cât și la degradarea siguranței fizice a operatorilor și a echipamentelor.

Spre deosebire de sistemele informatice domestice, numeroase protocoale utilizate în medii industriale au fost proiectate pentru rețele izolate și nu includ mecanisme moderne de securitate, precum autentificarea utilizatorilor la comunicare, criptarea datelor transferate sau filtrarea informațiilor primite. Pe de altă parte, studii recente evidențiază o creștere semnificativă și constantă a numărului de echipamente industriale expuse la Internet și a activităților de scanare și identificare a acestora. În acest context, monitorizarea și controlul comunicațiilor echipamentelor devin soluții esențiale reducerii suprafeței de atac și identificarea rapidă a conexiunilor neautorizate.

În mediul industrial, caracterul predictibil al naturii de comunicare reprezintă, din perspectiva securității, un avantaj care permite simplificarea complexității specifice soluțiilor altor medii. În multe scenarii, relațiile de comunicare dintre echipamente sunt bine definite, modificându-se rar pe parcursul existenței sistemelor. Această caracteristică importantă duce la posibilitatea identificării ușoare a comunicațiilor ce se abat de la funcționarea normală a aplicațiilor și permite implementarea unor mecanisme de control bazate pe politici explicite, în care comunicațiile autorizate sunt definite explicit, iar abaterile pot fi identificate și tratate imediat.

Lucrarea suport propune dezvoltarea unui sistem de securitate destinat monitorizării și controlului comunicațiilor în rețele industriale. Soluția utilizează un mecanism de filtrare

implementat la nivel *kernel*, capabil să analizeze și să filtreze traficul de rețea și să aplice politici de acces definite de administratorul sistemului. Sistemul permite atât raportarea automată a evenimentelor care nu respectă politicile impuse, cât și rejectarea conexiunilor cu alte elemente ale sistemului..

Pentru o administrare centralizată a infrastructurii rețelei, soluția include o componentă de server și o aplicație desktop care permit vizualizarea evenimentelor de securitate, configurarea în timp real a dispozitivelor și analiza conexiunilor din rețea.

Prezentarea domeniului

Sisteme industriale de control

Sistemele industriale de control (ICS) reprezintă un ansamblu de componente hardware și software utilizate pentru monitorizarea, coordonarea și controlarea proceselor petrecute în mediul industrial. Aceste elemente sunt introduse pentru a oferi personalului sistemului o imagine cât mai apropiată de starea reală și a modului de funcționare al sistemului, de cele mai multe ori adăugate în jurul infrastructurilor considerate critice. Sistemele ICS sunt o componentă fundamentală infrastructurilor actuale și sistemelor industriale de producție. Acestea sunt utilizate în momentul de față în numeroase domenii, precum: producția industrială, rafinarea petrolului, infrastructuri de apă și canalizare, sisteme de automatizare a clădirilor, distribuția și transportul de energie electrică.

Din punct de vedere evolutiv, sistemele ICS au fost concepute inițial ca și sisteme izolate, operate în medii închise, în care securitatea fizică era considerată suficientă pentru a asigura protecția infrastructurii și activității. Această paradigmă presupunea o separare completă a sistemelor industriale de rețele externe.

Odată cu progresul tehnologic și apariția unei necesități de integrare a sistemelor industriale în infrastructuri moderne, această izolare a devenit o cerință secundară, ajungând să fie exclusă în multe scenarii. Ca și urmare a acestor noi nevoi, sistemele ICS au început să fie conectate la rețele corporative și, în numeroase cazuri, indirect la Internet.

Din punct de vedere al arhitecturii unui sistem industrial de control, acesta este compus din mai multe clase de echipamente, fiecare cu roluri bine definite în procesul de control și supraveghere. Dintre clasele cele mai des întâlnite se pot enumera:

- PLC (*Programmable Logic Controllers*)[1] - utilizate pentru implementarea logicii de control direct asupra echipamentelor fizice
- SCADA (*Supervisory Control and Data Acquisition*)[2] - responsabile pentru colectarea, centralizarea și vizualizarea datelor din procesul industrial

Aceste componente au funcționalități interconectate în rețele de comunicație industrială, formând astfel sisteme distribuite a căror schimb de informații, latență în comunicare și fiabilitate sunt factori critici.

În primele generații ale componentelor software în sisteme industriale, securitatea nu a reprezentat un obiectiv principal, iar accentul era pus în mod exclusiv pe funcționalitatea, stabilitatea și eficiența operațională.

Odată cu apariția conectivității IP în infrastructurile industriale, posibilitatea de atac a acestora a crescut în mod exponențial. Această tranziție a condus la apariția multor vulnerabilități, deoarece protocoalele industriale nu includ mecanisme native de securitate precum criptare sau autentificare.

În plus, integrarea sistemelor ICS cu rețele IT tradiționale a creat o nouă suprafață de atac, ce permite propagarea amenințărilor din mediul *enterprise* către infrastructura industrială.

Metode de comunicare utilizate

În contextul comunicației proceselor, între componentele sistemelor ICS, schimbul de date este realizat prin intermediul unor protocoale și standarde specializate, adaptate cerințelor specifice mediului industrial. Acestea facilitează comunicarea, schimbul de date între diferite tipuri de echipamente și asigură coordonarea funcționalităților între procesele de control și cele de monitorizare.

Unul dintre cele mai cunoscute protocoale utilizate este *Modbus*[3], introdus în 1979, care rămâne răspândit la scară largă în aplicații de automatizare industrială datorită simplității și interoperabilității ridicate. Acesta este utilizat frecvent pentru comunicarea dintre PLC-uri și echipamente de câmp, însă nu oferă protecție împotriva interceptării traficului.

În mod similar, *Siemens S7*[4] este un protocol utilizat în ecosistemul echipamentelor PLC produse de *Siemens*. Acesta este întâlnit în procese industriale complexe unde este necesară controlul în timp real a echipamentelor de producție.

DNP3 (*Distributed Network Protocol*)[5] reprezintă un protocol dezvoltat inițial pentru rețelele din America de Nord utilizat pe scară largă pentru comunicarea dintre centrele de control SCADA și echipamentele de teren din infrastructurile de distribuție a energiei electrice, a apei și a gazelor naturale. Comunicarea se realizează prin portul 2000, iar protocolul este proiectat să funcționeze eficient pe legături seriale cu lățime de bandă redusă și latențe mari, caracteristici des întâlnite rețelelor de transport pe distanțe mari. Versiunea clasică și răspândită a protocolului nu include mecanisme de autentificare sau criptare, dar există totuși o extensie dedicată, *DNP3 Secure Authentication*, care adaugă o verificare a integrității comenzilor transmise, însă aceasta nu este adoptată universal.

IEC 60870-5-104[5] este standardul european echivalent lui DNP3, utilizat pentru comanda și monitorizarea echipamentelor din rețelele electrice de transport și distribuție. Aceasta este o extensie a standardului serial IEC 60870-5-101 adaptat pentru transmiterea datelor prin rețea TCP/IP prin portul 2404 în formatul standard original al mesajelor. La fel ca și în cazul protocolului DNP3, comunicarea se realizează fără criptare și fără autentificarea participanților, vulnerabilitate adresată parțial de standardul complementar IEC 62351, care definește mecanismele de securitate aplicabile protocoalelor din familia *IEC 60870*.

Un alt protocol important este *BACnet*[6], utilizat în special în sistemele de management al clădirilor, precum control *HVAC*, iluminatului și securității fizice, frecvent implementat în infrastructuri comerciale industriale. Comunicarea se realizează prin portul 47808, dar sunt alocate opțional porturi suplimentare în intervalul 47809-47823 pentru rețele cu un număr mare de dispozitive sau pentru segmentarea traficului între mai multe subrețele *BACnet/IP*. În implementările tradiționale, protocolul nu include mecanisme native de protecție, ceea ce permite exploatarea prin enumerarea echipamentelor și punctelor de control disponibile prin mesaje de tip *broadcast*. Versiunile moderne ale acestui protocol, precum *BACnet Secure Connect*, introduc mecanisme de securitate bazate pe criptare și autentificare, însă nu sunt implementate pe scară largă.

EtherNet/IP[7] reprezintă o adaptare mai modernă a protocolului *Common Industrial Protocol* (CIP) pentru rețele *Ethernet* standard, dezvoltat și întreținut de organizația ODVA (*Open DeviceNet Vendors Association*). Acesta este utilizat extensiv în automatizarea industrială, în special în ecosistemul echipamentelor produse de *Rockwell Automation*, pentru comunicarea dintre PLC-uri, module de intrare/ieșire și dispozitive de câmp. Comunicarea este făcută prin portul 44818, utilizat pentru mesajele explicite de configurare și diagnosticare, și prin

porturile 2221 și 2222 pentru schimbul ciclic de date de proces. Similar altor protocoale industriale tradiționale, acest protocol nu integrează nativ mecanisme de securitate.

ANSI C12.22[8] reprezintă un standard dezvoltat pentru transportul datelor de telemetrie între contoarele electrice inteligente și sistemele centrale de gestiune a infrastructurii de măsurare avansată (*Advances Metering Infrastructure*), comunicare făcută prin protocolul 1153.

Un alt standard important este *IEC 61850*[5], un standard modern utilizat în automatizarea spațiilor electrice. Acesta este considerat mai avansat din punct de vedere al modelării datelor și interoperabilității între echipamente inteligente. Standardul definește un model orientat pe obiecte pentru reprezentarea dispozitivelor și funcțiilor printr-o stație electrică, independent de producător, incluzând mai multe protocoale de comunicare specializate.

Pentru o analiză comparativă a principalelor protocoale și standarde în sistemele industriale de control (ICS), tabelul următor [Tabelul 1 - Comparatie între principalele protocoale ICS (adaptat după [9])] evidențiază atât caracteristicile funcționale, cât și informații relevante din perspectiva securității cibernetice și a expunerii acestora în medii de comunicare moderne. Pentru fiecare mecanism de comunicație industrial sunt prezentate porturile standard, domeniile tipice de utilizare (precum automatizarea proceselor industriale, rețele electrice inteligente sau managementul clădirilor), precum și modalitățile prin care traficul asociat poate fi identificat și analizat. Acest tabel este preluat și adaptat după studiul realizat de M. Nawrocki *et al.* în lucrarea „*Uncovering Vulnerable Industrial Control Systems from the Internet Core*”[9] în care se analizează vizibilitatea și expunerea protocoalelor industriale în Internet. Lucrarea realizează o comparație între elementele responsabile de comunicare din perspectiva utilizării, identificării în trafic și securitatea lor.

Captarea traficului și identificarea este realizată cu ajutorul aplicației *Wireshark*, Aplicația *Wireshark* reprezintă unul dintre cele mai utilizate aplicații de captură și analiză a pachetelor de rețea, și permite recunoașterea protocoalelor prin intermediul unor module numite *disecatoare (dissectors)*, care interpretează conținutul pachetelor și afișează informațiile într-un format ușor de analizat. În tabel sunt precizate atât mecanismele de comunicație pentru care există suport de disecție normală (DN), bazată pe porturi sau identificatori expliți ai protocolului, cât și cele care beneficiază de disecție euristică (DE), metodă prin care protocolul este identificat pe baza structurii și caracteristicilor conținutului transportat, chiar și atunci când utilizează porturi diferite de cele precizate ca și standard.

De asemenea, tabelul include informații privind nivelul de suport oferit de platformele de scanare și monitorizare la scară largă, utilizate pentru identificarea serviciilor și dispozitivelor expuse la Internet. Aceste platforme includ *Censys* (C), specializată în cartografierea infrastructurilor conectate la Internet, *Shodan* (S), motor de căutare dedicat dispozitivelor și serviciilor expuse public, *Rapid7* (R), cunoscut pentru proiectele de cercetare și măsurare a expunerii serviciilor de rețea, și *Kudelski Security* (K), care dezvoltă și publică rezultate de scanare pentru diverse mecanisme de comunicație industrială.

Pentru fiecare mecanism este prezentat și numărul vulnerabilităților publice înregistrate în baza de date CVE (*Common Vulnerabilities and Exposures*), un standard utilizat pentru catalogarea și identificarea vulnerabilităților de securitate. Acest indicator oferă o perspectivă asupra gradului de analiză și expunere al protocolului pe toată perioada existenței acestuia. Deși un număr ridicat de CVE-uri nu reprezintă o măsură exactă a riscului curent, acesta evidențiază existența unor vulnerabilități descoperite de-a lungul timpului și poate indica gradul cu care protocolul a fost afectat de probleme de securitate în procesul de proiectare, implementare, utilizare sau adaptare.

Prin corelarea acestor informații, tabelul evidențiază caracteristicile mecanismelor de comunicație industriale tradiționale și cele considerate moderne, din perspectiva expunerii acestora la rețea.

Protocol / Standard	Porturi	Utilizare	Wireshark (DN/DE)	Autentificare	Criptare	Bytes până la identificare	Suport	CVE
Modbus	502	Automatizare	DN	Nu	Nu	74 B	C/S/K	23
Siemens S7	102	Automatizare	DE	Nu	Nu	93 B	C/S	7
EtherNet/IP	2221, 2222, 44818	Automatizare	DN	Nu	Nu	74 B	S	9
BACnet	47808–47823	Managementul clădirilor	DN	Parțial	Parțial	46 B	C/S/R/K	7
DNP3	20000	Rețele electrice	DN/DE	Parțial	Parțial	62 B	C/S	39
IEC 60870-5-104	2404	Rețele electrice	DN	Nu	Nu	76 B	S	0
ANSI C12.22	1153	Inspecția consumului de resurse	DN	Nu	Nu	N/A	-	0
IEC 61850 (MMS)	102	Rețele electrice	DN/DE	Parțial	Parțial	144 B	-	0

Tabelul 1 - Comparație între principalele protocoale ICS (adaptat după [9])

Legendă: DN - disecție normală; DE - disecție euristică; C – *Censys*; S – *Shodan*; R – *Rapid7*; K - *Kudelski Security*; CVE - *Common Vulnerabilities and Exposures*;

Expunerea sistemelor ICS

Sistemele industriale de control reprezintă ținte de interes pentru actori malițioși, atât din motive economice cât și din alte motive diverse. Compromiterea acestor sisteme poate duce la întreruperea proceselor industriale, deteriorarea și oprirea prematură a echipamentelor fizice sau punerea în pericol a persoanelor responsabile de operarea acestora. Din acest motiv, înțelegerea vectorilor de atac specifici mediilor ICS este esențială pentru proiectarea unor mecanisme eficiente de protecție.

Atacurile asupra sistemelor ICS vizează, în general, trei categorii de obiective: disponibilitatea sistemului, integritatea datelor și proceselor de control și confidențialitatea informațiilor operaționale. Spre deosebire de atacurile asupra sistemelor IT tradiționale, unde impactul este predominant la nivel informațional, atacurile asupra sistemelor ICS pot produce consecințe fizice asupra proceselor controlate.

Vulnerabilitățile specifice infrastructurilor ICS nu derivă exclusiv din deficiențele de proiectare ale protocoalelor de comunicare discutate în secțiunea anterioară, ci sunt de asemenea și consecințele unei liste stufoase de constrângeri operaționale caracteristice acestui mediu. Una dintre cele mai semnificative constrângeri este durata de viață extinsă a echipamentelor industriale, proiectate pentru cicluri de funcționare de zeci de ani. Această caracteristică implică faptul că infrastructurile ce se construiesc trebuie să ia în considerare durata de viață și suport oferit de serviciile de *firmware* al sistemelor de operare și cel de actualizări al aplicațiilor folosite. Se dorește ca aceste servicii să ofere suport și să fie disponibile pe toată durata de viață a activității procesului industrial. Întreruperea acestui suport duce la apariția de vulnerabilități nerezolvate, chiar dacă acestea ar fi cunoscute. Soluția la întâmpinarea acestui caz este schimbarea a serviciilor, care, din motive de performanță, pot fi factori decizionali în alegerea altor produse, ce duce nu doar la înlocuirea unui singur produs, ci la un grup, ce reprezintă o activitate greoaie.

Un alt factor îl reprezintă constrângerile de disponibilitate ale sistemelor industriale. Aplicarea unei corecții de securitate sau repornirea unui echipament presupune adesea operarea unui proces industrial critic, cu implicații economice sau riscuri operaționale directe. Din acest motiv, operatorii pot decide să amâne intervențiile de securitate pentru a fi executate la intervale fixe de timp și aplicarea acestora simultan, pentru a scade numărul de reporniri și al momentelor de indisponibilitate. Această metodă are ca și consecință prelungirea ferestrei de expunere la atacuri pentru care soluțiile sunt deja cunoscute și disponibile.

Nu în ultimul rând, accesul fizic la echipamente reprezintă o problemă distinctă comparativ cu mediile IT. Echipamentele industriale sunt amplasate adesea în locații distribuite geografic, unde supravegherea fizică poate fi limitată. Un atacator cu acces fizic la un PLC sau la un *switch* industrial poate conecta dispozitive de interceptare, introduce malware prin porturi USB sau modifica direct configurația echipamentului, ocolind complet mecanismele de securitate de rețea.

Tipuri de atacuri în sisteme ICS

Atacurile de tip *man-in-the-middle* (MitM)[10] explorează absența mecanismelor de autentificare și criptare din protocoalele industriale. Un atacator poziționat în rețeaua industrială poate intercepta și modifica comenzile transmise între stațiile de control și echipamente, fără ca acestea să detecteze interferența. Tehnici precum *ARP spoofing* permit atacatorilor să intercepteze comunicarea între sisteme și să redirecționeze traficul prin propriul echipament fără să întrerupă sau să fie detectat de stațiile de control. Astfel, operatorul poate fi dezinformat prin intermediul unei stații care primește valori artificiale, în timp ce echipamentele fizice sunt modificate de atacatori.

Atacurile de tip *replay*[10] constau în capturarea unor secvențe de comunicații reale și retransmiterea ulterioară a lor pentru a produce efecte nedorite. Un atacator poate înregistra comenzile asociate unei operații legitime și le poate retrimite în momente nepotrivite pentru a determina echipamentele să execute anumite acțiuni fizice. Această categorie este periculoasă deoarece nu este obligatorie înțelegerea secvenței înregistrare. În cazul în care singura cerință a atacatorului este perturbarea sistemelor, semantica protocoalelor vizate este opțională, iar simpla înregistrare și retransmitere a comenzilor va crea rezultatul dorit.

Injectarea de comenzi[10] reprezintă una dintre cele mai grave forme de atac în mediile ICS. Deoarece protocoalele industriale clasice nu verifică sursa comenzilor primite, orice entitate cu acces la rețeaua industrială poate emite instrucțiuni arbitrare direct către echipamentele de control. Consecințele acestui tip de atac pot include modificarea parametrilor de funcționare a proceselor fizice, activarea sau dezactivarea sistemelor de siguranță sau forțarea echipamentelor în stări operaționale periculoase.

Atacurile de tip *denial-of-service* (DoS)[10] vizează perturbarea disponibilității sistemelor de control prin suprasolicitarea resurselor de comunicare sau de procesare a echipamentelor. Datorită resurselor limitate ale dispozitivelor industriale și a sensibilității lor la latență, chiar

și atacuri de mică intensitate pot cauza efecte grave ale funcționalității sistemelor. În mediul industrial întreruperea comunicației de control poate fi echivalentă cu oprirea unui proces de producție sau, în cazuri mai grave, cu crearea unor situații de risc a siguranței operatorilor.

Atacurile asupra ***firmware-ului și software-ului de control***[11] vizează modificarea logicii de execuție a echipamentelor industriale. Prin exploatarea unor vulnerabilități în mecanismele de actualizare a *firmware*-ului sau prin accesul direct la interfețele de programare, un atacator poate introduce cod malițios care alterează comportamentul echipamentului în mod persistent. Modificările pot rămâne active chiar și după repornirea echipamentului și pot fi dificil de identificat la inspecție.

Atacurile de tip ***lateral movement***[12] descriu atacurile prin care un atacator, după obținerea unui punct inițial de acces, traversează progresiv rețeaua pentru a ajunge la sisteme de interes mai mare. Un scenariu întâlnit frecvent presupune compromiterea inițială a unui sistem din rețeaua IT corporativă, urmată de o exploatare a punctelor de legătură dintre rețeaua IT și rețeaua operațională industrială (*OT – Operational Technology*), pentru a pătrunde ulterior în infrastructura industrială. Odată ajuns în rețeaua OT, atacatorul beneficiază de o suprafață de atac semnificativ mai mare, datorită numărului ridicat de echipamente cu mecanisme de securitate reduse sau absente.

Atacurile asupra **lanțului de aprovizionare**[13] vizează compromiterea *software*-ului sau hardware-ului înaintea ajungerii acestuia la operatorul final. Prin introducerea de cod malițios în actualizările de *firmware*, în bibliotecile software utilizate de platformele de control sau chiar în componente hardware, atacatorii pot obține acces persistent în infrastructura victimei fără a exploata vreo vulnerabilitate de rețea. Natura acestei categorii de atac o face deosebit de dificil de detectat prin metodele clasice de verificare, deoarece sursa compromiterii se află în afara ariei de securitate a organizației vizate.

Cercetările prezentate în cadrul conferinței BlackHat[14][15] au contribuit semnificativ la conștientizarea vulnerabilităților practice ale sistemelor industriale. Prezentările au demonstrat și explicat metode de exploatare care, pentru a fi duse la bun sfârșit, nu a fost nevoie de un nivel de expertiză excepțională din partea unui atacator.

Meixell și Former[14] demonstrează că sistemele SCADA pot fi compromise prin tehnici accesibile, pornind de la premisa că protocoalele industriale, prin lipsa mecanismelor de autentificare, tratează implicit orice sursă din rețea ca fiind de încredere. Lucrarea prezintă scenarii concrete de exploatare în care un atacator cu acces la rețeaua industrială poate citi și modifica registrele unui PLC, poate forța stări nedorite ale ieșirilor fizice și poate perturba

procesele controlate, fără a lăsa urme semnificative în jurnalele sistemului. O concluzie importată a lucrării este că prezența unui sistem SCADA nu adaugă în mod implicit protecție sistemului comparativ cu comenzile directe emise la nivel de protocol, întrucât valorile afișate operatorilor pot fi separate de starea reală a echipamentelor de câmp. Autorii subliniază că securitatea prin obscuritate, pe care se bazau implicit multe implementări SCADA, nu mai reprezintă o protecție suficientă în contextul conectivității moderne, iar simpla cunoaștere a structurii unui protocol industrial, informație care în multe cazuri este disponibilă public, este suficientă pentru a crea un atac funcțional.

Klick et al.[15] extind această idee prin analiza echipamentelor PLC expuse direct la Internet. Aceștia demonstrează că echipamentele PLC pot fi utilizate similar *backdoor*-urilor clasice din domeniul securității cibernetice. Autorii identifică un număr semnificativ de PLC-uri ce pot fi accesate public și arată cum acestea pot fi folosite ca și puncte de intrare în infrastructurile industriale. Din acea poziție au acces la elementele de control ale echipamentului și, prin urmare, pot utiliza aceste componente ca bază în atacarea sau compromiterea altor dispozitive din rețeaua internă. Lucrarea evidențiază că multe dintre aceste echipamente rulează servicii de acces la distanță cu credențiale implicite sau fără autentificare, iar interfețele de programare ce sunt expuse permit modificarea logicii de control fără a exista restricții de acces. Un aspect deosebit de îngrijorător semnalat de autori este că un atacator extern poate obține, prin exploatarea unui PLC expus la Internet, un punct de prezență în rețeaua internă industrială, de unde se pot iniția atacuri suplimentare împotriva altor echipamente inaccesibile direct din exterior. Această realitate transformă PLC-urile expuse la Internet din simple echipamente de autorizare în potențiale puncte de compromitere a întregii infrastructuri industriale, indiferent de nivelul de protecție al perimetrului de rețea.

Cel mai notabil atac documentat asupra unui sistem industrial rămâne *Stuxnet*[16], descoperit în 2010, care a vizat centrifugele de îmbogățire a uraniului din Iran. *Malware*-ul a exploatat vulnerabilități din sistemele Windows pentru a ajunge la PLC-urile *Siemens S7* care controlau centrifugele și s-a fructificat prin modificări subtile a unor parametrii de funcționare. Atacul a demonstrat că sistemele industriale pot fi compromise în mod precis și silențios, cu efecte importante la nivelul integrității fizice a echipamentelor, fără ca operatorii să detecteze anomalia prin mijloacele de monitorizare disponibile. *Stuxnet* rămâne primul caz documentat public în care acest tip de exploatare a devenit armă cibernetică, cu impact fizic direct, și a redefinit percepția asupra amenințărilor la adresa infrastructurilor critice.

Atacurile asupra rețelei electrice din Ucraina[17] din decembrie 2015 reprezintă de asemenea un exemplu documentat relevant de compromitere a sistemelor ICS cu impact direct asupra infrastructurii critice. În cadrul acestui incident, mai multe companii de distribuție a energiei au fost afectate simultan. Acestea au rezultat în întreruperea alimentării de energie pentru aproximativ 225.000 de consumatori și necesitatea trecerii echipamentelor la moduri manuale de operare. Investigarea acestui eveniment arată că atacatorii au obținut în faza inițială acces la rețelele informatice ale organizațiilor prin campanii de tip *spear-phishing*. Ca și elemente utilizate, au fost descoperite documente *Office* malițioase care instalau *malware* pentru a compromite sistemele interne. Ulterior, aceștia au realizat escaladarea privilegiilor, iar cu ajutorul credențialelor acumulate și prin deplasarea laterală către rețeaua internă, au ajuns la interacțiunea directă cu stațiile de lucru ale operatorilor SCADA și sistemele de tip HMI. În faza finală a atacului, au fost executate comenzi directe asupra echipamentelor din infrastructura de distribuție, ce a condus la deschiderea întreruptoarelor din mai multe stații de transformare și la deconectarea unor porțiuni semnificative din rețea. În paralel cu acestea au fost utilizate mecanisme suplimentare de sabotaj, precum distrugerea sistemelor de operare prin *malware* de tip *wiper* ce șterg memoria unităților, compromiterea unor echipamente de comunicație și blocarea canalelor de suport prin atacuri telefonice de tip *denial-of-service (DoS)*. Acest incident a pus în evidență faptul că atacurile asupra sistemelor de tip SCADA pot avea în vizor procesele fizice de control ale sistemelor și demonstrează că un atac cibernetic bine coordonat poate produce efecte majore unei infrastructuri electrice, cu impact asupra populației și a economiei.

Descrierea problemei și soluția propusă

O cerință esențială a mediului industrial cu privire la soluțiile de securitate o reprezintă capacitatea de monitorizare a dispozitivelor în cadrul unei rețele locale. Totodată, delimitarea traficului și blocarea sau raportarea abaterilor de la natura definită sunt cerințe cu aceeași valoare de lucru. Se dorește un nivel cât mai mare de control asupra conexiunilor ce pot fi făcute între dispozitivele sistemului.

Monitorizarea și filtrarea traficului dispozitivelor în cadrul unei rețele industriale implică analiza detaliată a pachetelor de date și identificarea comportamentelor anormale sau care nu se conformează cu politicile de comunicare stabilite.

Soluții existente de securitate a sistemelor ICS

Soluțiile de securitate destinate mediilor ICS pot fi clasificate, în funcție de mecanismul de detecție utilizat, în trei categorii principale: soluții bazate pe semnături, soluții bazate pe detecția anomaliilor și soluții bazate pe politici de acces explicit. Fiecare dintre aceste abordări prezintă avantaje și limitări specifice, iar alegerea uneia dintre ele este determinată de contextul operațional, de resursele disponibile și de natura amenințărilor vizitate.

Soluții bazate pe semnături

Sistemele bazate pe semnături, precum *Snort* și *Suricata* [18] funcționează prin compararea traficului de rețea cu o bază de date de tipare cunoscute asociate atacurilor documentate. Atunci când un pachet sau o secvență de pachete corespunde unui tipar existent, sistemul generează o alertă sau blochează comunicarea. Această abordare oferă o rată redusă de *fals-positiv* pentru amenințări cunoscute și poate fi implementată fără o fază de antrenament prealabilă. Cu toate acestea, limitarea fundamentală a acestei categorii constă în incapacitatea de a detecta atacuri noi sau variante ale atacurilor existente pentru care nu a fost definită o semnătură corespunzătoare. Folosind acest tip de identificare, acuratețea și performanța de funcționare se bazează în întregime pe baza de date folosită. Datorită naturii modulare și complementare a soluției, aceasta poate fi utilizată ca și complementare a altor soluții, pentru a crea o metodă mai completă și complexă. Din punct de vedere a naturii de creare și actualizare a bazelor de semnături, în contextul mediilor ICS, unde protocoalele sunt specializate și atacurile pot fi construite pe baza caracteristicilor specifice ale acestora, aceasta reprezintă o provocare continuă.

Soluții bazate pe detecția anomaliilor

Direcția predominantă a soluțiilor de securitate recente a sistemelor ICS este cea a detecțiilor anomaliilor prin metode statistice și de învățare automată. Această abordare pornește de la premisa că traficul normal dintr-o rețea industrială prezintă tipare stabile și predictibile, iar abaterile de la aceste tipare pot indica prezența unui atac sau a unei disfuncționalități.

T. K. Das et al.[19] propun utilizarea metodei *Logical Analysis of Data* (LAD) pentru detectarea anomaliilor în sistemele ICS. Metoda construiește un model al comportamentului normal pe baza datelor operaționale istorice, ce identifică tipare logice care caracterizează funcționalitatea corectă a sistemului. Abaterile de la aceste tipare sunt semnalizate ca potențiale incidente de securitate. Avantajul acestei abordări constă în capacitatea ridicată de înțelegere a modelului generat, care poate fi proporțională cu gradul cu care datele folosite în

etapa de antrenare descriu cât mai corect sistemul real. Cu toate acestea, comportamentele anormale care nu se abat suficient de mult de la tiparele normale pot să rămână nedetectate.

J. Kim, J.-H. Yun și H. C. Kim[20] propun o arhitectură bazată pe rețele neuronale de tip *sequence-to-sequence* pentru detectarea anomaliilor în traficul sistemelor ICS. Modelul este antrenat exclusiv pe date normale și învață să reconstruiască secvențe de comunicații din rețeaua industrială. În faza de detecție, modelul compară secvențele observate cu cele pe care le reconstruiește pe baza comportamentului învățat în timpul antrenării. O eroare ridicată de reconstrucție indică faptul că secvența analizată se diferențiază semnificativ de tiparele normale și este semnalată ca și o anomalie. Această abordare este suficientă în detectarea deviațiilor subtile față de tiparele normale, și acoperă atacurile care nu generează schimbări bruște în trafic, ci alterează gradual comportamentul procesului controlat. Limitarea principală rămâne, însă, problema generală a soluțiilor bazate pe anomalii, și anume dependența de calitatea și diversitatea datelor de antrenament, precum și sensibilitatea la variațiile naturale ale procesului industrial, care pot genera alarme false.

M. Elnour et al.[21] prezintă un cadru de detectare și protecție în timp real a atacurilor de tip injectare de date false la nivelul senzorilor prin utilizarea unui ansamblu de tehnici de învățare automată. Lucrarea adresează o categorie specifică de atac în care valorile raportate de senzori sunt modificate pentru a induce în eroare sistemul de control, fără a perturba direct comunicațiile de rețea. Soluția propusă combină mai mulți clasificatori pentru a obține o rată de detecție ridicată și un număr redus de fals-pozitive, integrând totodată un mecanism de răspuns automat care poate corecta valorile compromise înainte ca acestea să influențeze deciziile de control. Deși eficientă pentru scenariul specific vizat, această abordare necesită resurse computaționale semnificative și o fază extinsă de antrenament și validare de date reprezentative pentru instalația monitorizată.

L. Wang[22] extinde direcția bazată pe învățare automată prin introducerea tehnicilor de adaptare de domeniu pentru detectarea intruziunilor în scenarii în care distribuția traficului variază între diferite instalații sau configurații ale sistemului. Lucrarea combină metode de *clustering* cu tehnici de *transfer learning* pentru a permite aplicarea unui model antrenat pe datele unei instalații la o instalație diferită, ceea ce reduce costul asociat colectării și etichetării datelor de antrenament pentru fiecare nouă implementare. Această abordare răspunde unei limitări practice semnificative a soluțiilor bazate pe anomalii, și anume costul ridicat al construirii unui model specific pentru fiecare mediu industrial în parte.

Soluții bazate pe politici

Spre deosebire de abordările descrise anterior, soluțiile bazate pe politici de acces explicit nu încearcă să deducă comportamentul normal prin analiză statistică sau modele probabilistice, ci definesc în mod direct și clar comunicațiile autorizate în cadrul rețelei. Orice comunicare care nu se regăsește în lista celor explicit permise este tratată ca și anormală, indiferent dacă prezintă sau nu caracteristicile statistice a unei activități normale.

Această abordare se aliniază natural cu caracteristicile generale ale mediului industrial, unde relațiile de comunicare dintre echipamente sunt, în general, bine definite și stabile pe perioade lungi de timp. Natura deterministă și predictibilă a traficului industrial face ca definirea explicită a conexiunilor permise să fie nu doar fezabilă, ci și mai precisă decât modelarea statistică a funcționalității legitime.

Sistemul propus în lucrarea de față adoptă această filozofie, și implementează un mecanism de filtrare la nivel *kernel* ce aplică politici de acces definite explicit de administratorul rețelei. Fiecare conexiune este verificată față de o listă de adrese și porturi autorizate, iar în funcție de modul de funcționare configurat, conexiunile neautorizate sunt raportate și blocate. Această abordare oferă un set de avantaje distincte față de soluțiile bazate pe detecția anomaliilor.

În primul rând, sistemul nu necesită o fază de antrenament prealabilă pe date operaționale. Prin aceasta elimină dependența de seturi de date reprezentative, care pot fi dificil de obținut în medii industriale reale. Pe lângă construcția manuală a listei de conexiuni permise, soluția oferă un mecanism alternativ de construire progresivă a listei, în care se adaugă dispozitivele ce participă la traficul observat în rețea pentru o perioadă controlată de timp, fără a implica însă modele probabilistice sau praguri de decizie arbitrar (modului de funcționare MONITORING al sistemului).

În al doilea rând, absența ambiguității specifice modelelor statistice elimină categoria fals-positivelor generate de variațiile naturale ale procesului industrial. O conexiune este sau nu este permisă în mod explicit, fără a exista o zonă gri de incertitudine care să necesite o calibrare continuă a pragurilor de detecție.

În al treilea rând, implementarea filtrării la nivel *kernel* oferă o latență de procesare redusă și o integrare profundă în stiva de rețea a sistemului de operare, prin care se permite nu doar semnalizarea conexiunilor neautorizate, ci și blocarea efectivă a acestora. Soluțiile bazate pe detecția anomaliilor operează, în general, pasiv, oferind alerte *post-factum*, fără o capacitate reală de a preveni comunicarea neautorizată, fapt ce se datorează posibilității continue de apariții a fals-positivelor.

Pentru o sintetizare comparativă a principalelor caracteristici ale categoriilor de soluții discutate, a fost creat tabelul 2:

Soluție	Detectează atacuri necunoscute	Blochează trafic	Necesită antrenament	Cantitate de <i>fals-pozitive</i>
Bazată pe semnături (Snort, Suricata)	NU	DA	NU	Reduse
Bazată pe anomalii – LAD	DA	NU	DA	Medii
Bazată pe anomalii – Seq2Seq	DA	NU	DA	Medii
Bazată pe anomalii – ML	DA	NU	DA	Reduse
Bazată pe anomalii – Transfer learning	DA	NU	DA	Medii
Bazată pe politici (sistemul propus)	Parțial	DA	NU	Absente

Tabelul 2 - Comparatie solutii diferite de detectie

Limitarea principală a abordării bazate pe politici constă în incapacitatea de a detecta atacuri care se produc în cardul conexiunilor autorizate, precum injectarea de comenzi false provenite de la o sursă legitimă sau modificarea valorilor transmise în cardul unui protocol permis. Această categorie de amenințare este cel mai bine acoperită de soluțiile bazate pe detecția anomaliilor, care analizează conținutul și comportamentul comunicațiilor, nu doar identificarea participanților. Prin urmare, cele doua abordări nu se exclud reciproc, ci se completează, o arhitectură de securitate cuprinzătoare putând beneficia de combinarea mecanismelor de control al accesului bazate pe politici cu tehnici de inspecție mai profundă a traficului autorizat.

Prezentarea aplicației

Soluția propusă cuprinde o aplicație desktop concepută pentru administrarea centralizată a sistemului. Pe baza raportărilor făcute, administratorul sistemului poate avea o imagine de ansamblu a traficului produs în rețea și a regulilor de funcționare a fiecărui dispozitiv. Aplicația permite identificarea rapidă a elementelor ce inițiază conexiuni ce se abat de la profilele setate, indiferent dacă modul de funcționare setat blochează sau permite aceste conexiuni, și facilitează modificarea dinamică a configurațiilor dispozitivelor din rețea, atât a modului de lucru cât și a modificării listei dispozitivelor permise.

Comportamentul fiecărui echipament monitorizat este definit de unul dintre cele patru moduri de funcționare disponibile:

- **LISTENING** – aplicația rulează în mod permisiv, în care se acceptă conexiunile ce nu se regăsesc în lista celor permise, dar le raportează. Acest mod de funcționare este util în special în faza inițială de observare a rețelei, atunci când administratorul dorește să înțeleagă tiparele de comunicare existente fără a perturba funcționalitatea sistemului.
- **REACTIVE** – rulează în modul strict, refuzând conexiunile care nu sunt găsite în lista celor permise și raportând fiecare încercare de comunicare blocată. Acest mod este recomandat sistemelor ce rulează în producție, iar politica de acces este deja definită și validă.
- **DISABLED** – aplicația permite orice conexiune, iar raportarea evenimentelor este dezactivată complet. Acest mod este destinat scenariilor când monitorizarea și filtrarea trebuie suspendate temporar, fără a elimina componentele de filtrare aflate pe echipamentele sistemului.
- **MONITORING** – conexiunile detectate sunt adăugate automat în lista conexiunilor permise, și se construiește astfel în mod dinamic politica de acces pe baza traficului observat. Acest mod este util pentru definirea inițială a relațiilor de comunicare autorizate, în care o abordare automată este posibilă și practică configurării sistemului.

Aplicația utilizează o arhitectură modulară, compusă din componente cu responsabilități bine delimitate. Separarea clară a responsabilităților între componente permite o dezvoltare și testare independentă, fără a afecta funcționarea celorlalte. Aceste componente sunt:

1. **componenta de filtrare la nivel *kernel***, responsabilă de interceptarea pachetelor, extragerea informațiilor relevante, transmiterea acestora pentru o inspecție generală și blocarea pachetelor definite în fișierul de configurație.
2. **componenta de intermediere**, responsabilă de transmiterea informațiilor între modulul *kernel* și componenta centrală. Acesta este de asemenea responsabilă de citirea fișierelor de configurații; sincronizarea acestora cu informațiile primite de la server și trimiterea noilor conexiuni ce vor fi permise către modul *kernel*.
3. **componenta centrală**, responsabilă de stocarea tuturor informațiilor primite de la dispozitivele monitorizate, de gestionarea bazei de date ce memorează traficul sistemului, și de intermedierea comunicării dintre echipamente și componenta de administrare.
4. **componenta de administrare**, ce oferă posibilitatea de vizualizare a conexiunilor ce au avut loc pe dispozitivele monitorizare și modificarea configurațiilor acestora la nivel individual sau global.

Comunicarea dintre componentele care rulează pe echipamentul monitorizat și componenta centrală se realizează prin intermediul unei interfețe de tip REST API (*Representational State Transfer Application Programming Interface*), un stil arhitectural utilizat pe scară largă pentru dezvoltarea serviciilor distribuite. Un API de acest tip permite schimbul de informații între aplicații prin intermediul protocolului HTTP, cu ajutorul operațiilor standard GET, POST, PUT și DELETE pentru interogarea, transmiterea, modificarea și ștergerea datelor. Acest model facilitează interogarea dintre componentele dezvoltate independent.

În cadrul sistemului propus, componentele care rulează pe echipamentul monitorizat transmit către componenta centrală informații referitoare la rezultatele proceselor de monitorizare. La rândul său, componenta centrală expune o serie de *endpoint*-uri REST prin intermediul cărora datele pot fi recepționate, procesate și stocate. Componenta de administrare utilizează aceeași interfață pentru interogarea stării sistemului, vizualizarea informațiilor colectate și transmiterea modificărilor de configurare inițiate de administrator.

Informațiile trimise de echipamente, alături de configurația curentă a acestora, sunt stocate în într-o bază de date *SQLite*, pentru a oferi flexibilitate și posibilitatea analizării în detaliu a conexiunilor anterioare, precum și a modificărilor efectuate asupra configurației. Acest proces este gestionat de către componenta centrală.

Detalii de implementare

Aplicația este organizată în trei directoare principale, fiecare dintre acestea corespunde unei componente distincte sistemului. Primul director conține elementele care rulează extern echipamentului monitorizat, al doilea director include logica **componentei de intermediere** (*daemon*), iar ultimul este destinat **componentei de filtrare la nivel kernel**.

În directorul elementelor externe se regăsește atât **componenta de administrare** (aplicația *desktop*), cât și **componenta centrală** responsabilă de gestionarea și coordonarea sistemului [Figura 1 - Directorul de lucru].

```
./daemon  
./document  
./external  
./kernel_module  
  
./external/admin_application/  
./external/server/
```

Figura 1 - Directorul de lucru

Componenta de filtrare la nivel *kernel*

Această componentă reprezintă un modul *kernel* scris în limbajul de programare **C**, ce rulează în spațiul cu permisiuni elevate. Spre deosebire de programele obișnuite care rulează în spațiul utilizatorului și accesează resursele sistemului prin apeluri de sistem, un modul *kernel* este încărcat direct în spațiul de adresare al *kernelului* și beneficiază astfel de accesul imediat la structurile interne ale sistemului de operare și la interfețele de rețea. Această caracteristică oferă componentei atât performanță superioară în procesarea pachetelor, cât și un nivel de control ridicat, ce nu poate fi obținut din spațiul utilizatorului.

Dezvoltarea modulelor *kernel* în *Linux* impune respectarea unor constrângeri structurare. Modulul nu funcționează pe baza bibliotecii standard **C**, ci favorizează utilizarea funcțiilor și a *macro*-comenzilor expuse de *kernel* prin *header*-urile sale specifice. Gestionarea memoriei se face prin funcții specializate precum *kmalloc* și *kfree*, care operează în spațiul de adresare al *kernelului*. Un aspect important dezvoltării în acest spațiu reprezintă utilizarea operațiilor blocante, deoarece acestea pot duce la *lock*-uri al firului de execuție, ce va afecta stabilitatea întregului sistem.

Principala caracteristică a acestei componente constă în definirea și înregistrarea unui mecanism de interceptare a pachetelor (*network hook*) prin cadrul software *Netfilter*[23]. *Netfilter* reprezintă un subsistem al *kernelului Linux*, introdus în versiunea 2.4, care oferă un cadru arhitectural specializat pentru implementarea unor funcționalități avansate de rețea, cum ar fi interceptarea, inspectarea și blocarea pachetelor în anumite puncte specifice procesului de procesare. Cu ajutorul acestui cadrul software este permisă înregistrarea unor funcții pentru reapelare, care sunt invocate automat de către *kernel* la momentul procesării fiecărui pachet.

Arhitectura *Netfilter* definește cinci puncte principale de interceptare pentru traficul IPv4, fiecare corespunde unei etape distincte din procesul de rutare al pachetelor:

- `NF_INET_PRE_ROUTING` este primul punct atins de pachetele primite, înainte ca acestea să fie afectate de decizia de rutare. În acest punct se pot primi pachete ce nu au ca și destinație dispozitivul suport, sau pachetele sunt destinate a fi „consumate” în spațiul *kernel*, și nu în spațiul utilizatorului. Această divizare a pachetelor se face prin decizia de rutare.
- `NF_INET_LOCAL_IN` este atins de pachetele destinate proceselor locale de pe sistemul curent, după decizia de rutare.
- `NF_INET_FORWARD` reprezintă punctul de intermediere pentru pachetele care sunt rutate prin sistem spre o altă destinație din rețeaua locală. Acestea nu intră în spațiul utilizatorului, ci sunt transmise mai departe de către *kernel*.
- `NF_INET_LOCAL_OUT` este atins de pachetele generate de procesele locale, înainte de ieșirea din sistem.
- `NF_INET_POST_ROUTING` reprezintă ultimul punct înainte ca pachetele să fie trimise pe interfața de rețea.

Pentru scopurile sistemului de față, *hook*-ul `NF_INET_LOCAL_IN` reprezintă punctul optim de interceptare, deoarece permite inspecția tuturor pachetelor destinate echipamentului monitorizat, inclusiv a celor provenite de la alte dispozitive din rețeaua locală. În acest punct, decizia de rutare a fost deja luată, dar pachetul nu a ajuns încă la aplicația destinată, oferind posibilitatea de a bloca conexiunile neautorizate înainte ca acestea să fie procesate de aplicațiile utilizatorului.

În memoria componentei se află în mod constant lista conexiunilor permise, alături de modul curent de rulare. Lista este implementată ca o structură de date reziduală „*rhashtable*” în memoria *kernelului*, accesibilă rapid la procesarea fiecărui pachet nou. Protecția accesului

concurrent la această structură, necesară deoarece funcția introdusă ca și *hook* poate fi invocată simultan de mai multe nuclee ale procesorului, este asigurată prin utilizarea unui mecanism de sincronizare *mutex*.

La recepția unui pachet, funcția „*packet_hook*” extrage adresa IP sursă și portul destinație, verificând dacă perechea corespunde uneia dintre intrările din lista conexiunilor permise. Dacă pachetul provine de la adresa IP a serverului, acesta este acceptat necondiționat, pentru a permite comunicarea cu componenta centrală indiferent de configurația curentă. În funcție de rezultatul verificării și de modul curent de funcționare, funcția returnează una dintre valorile definite de *Netfilter*: *NF_ACCEPT*, pentru a permite continuarea procesării pachetului, sau *NF_DROP*, pentru a fi eliminată silențios. Înainte de returnarea deciziei, informațiile esențiale ale pachetului sunt extrase și plasate într-un *buffer* intern [Figura 2 - Structura informațiilor pachetelor] pentru a fi transmise componentei de intermediere.

În momentul în care se face cererea modificării listei de conexiuni sau a modului de funcționare, componenta de intermediere va informa modulul *kernel* de noua configurație, urmând ca acestea să fie salvate pe disc pentru restaurare la repornirea sistemului.

Comunicarea între modulul *kernel* și componenta de intermediere se face prin intermediul unui ***Miscellaneous character device***, care oferă posibilitatea folosirii unui *character device* într-un mod simplificat. Un *character device* este un tip de fișier specializat care permite comunicarea bidirecțională între spațiul *kernel* și cel al utilizatorului prin operațiile standard de citire și scriere. Clasa *miscellaneous* simplifică înregistrarea unui astfel de dispozitiv, prin eliminarea necesității alocării unui număr major de dispozitiv și prin reducerea codul necesar funcționării. Dispozitivul este expus în sistemul de fișiere sub calea „*/dev/...*”, de unde componenta de intermediere îl deschide și îl monitorizează în mod continuu.

Formatul datelor transmise prin intermediul *character device*-ului este unul binar, bazat pe structuri *C* de dimensiune fixă, transferate prin apeluri *ioctl()*. Fiecare eveniment este serializat într-o structură ce conține informațiile esențiale necesare analizei traficului [Figura 2 - Structura informațiilor pachetelor]. Structura include adresa IP sursă (*saddr*) și adresa IP destinație (*daddr*) utilizate pentru identificarea sistemelor ce participă la comunicare, precum și portul sursă (*sport*) și portul destinație (*dport*), care permit determinarea serviciilor sau aplicațiilor implicate în schimbul de date. Câmpul *protocol* indică protocolul de transport folosit, precum TCP, UDP sau ICMP, iar câmpul *ttl* (*Time to Live*) oferă informații utile despre ruta parcursă de pachet și poate fi utilizat în anumite tehnici de detecție. Structura conține și câmpul *total_len*, care reprezintă dimensiunea totală a pachetului IP, precum și

câmpul *tcp_flags*, utilizat pentru stocarea indicatorilor specifici protocolului TCP (în cazul de utilizare a acestui protocol) , cum ar fi SYN, ACK, FIN sau RST.

Pe lângă informațiile de nivel rețea și transport, structura include și numele interfeței de rețea prin care a fost observat pachetul (*inde*v), ceea ce permite identificarea punctului de intrare în sistem. De asemenea, sunt stocate adresa MAC sursă (*src_mac*) și adresa MAC destinație (*dst_mac*), informații utile pentru analiza comunicațiilor la nivelul legăturii de date și identificarea dispozitivelor din aceeași rețea locală, a căror adresă IP se poate schimba în timp. Toate aceste câmpuri sunt reprezentate într-o structură cu dimensiune fixă, ceea ce simplifică transferul datelor din spațiul *kernel* în spațiul utilizator și elimină necesitatea utilizării unor mecanisme suplimentare de parcurgere sau delimitare a informațiilor.

```
struct packet_info {  
    __be32 saddr, daddr;  
    __be16 sport, dport;  
    u8 protocol;  
    u8 ttl;  
    u16 total_len;  
    u8 tcp_flags;  
    char indev[IFNAMSIZ];  
    u8 src_mac[ETH_ALEN];  
    u8 dst_mac[ETH_ALEN];  
    struct list_head list;  
};
```

Figura 2 - Structura informațiilor pachetelor

Componenta de intermediere

Această componentă reprezintă un agent local *daemon* scris în limbajul C++ ce rulează în spațiul utilizatorului. Alegerea limbajului C++ pentru implementare a fost motivată de necesitatea unui nivel de abstractizare mai ridicat față de C, ce nu sacrifică performanța aplicației. Biblioteca standard C++ oferă spații de stocare generice precum *std::vector* și *std::deque*, care simplifică gestionarea datelor. Întreaga logică a agentului este cuprinsă în clasa *ConfigDaemon*, iar programul principal înregistrează funcții pe post de „cârlige” pentru semnalele *SIGINT* și *SIGTERM* pentru a permite o oprire controlată atât la întrerupere manuală, cât și la oprirea prin *systemctl*.

Agentul are două responsabilități de prioritate egală: transmiterea informațiilor extrase de la componenta de filtrare către componenta centrală, și informarea componentei de filtrare asupra noilor configurații și reguli modificate sau adăugate de administrator prin interfața de administrare. Datorită acestora, *daemon*-ul creat reprezintă elementul de sincronizare între starea dorită și impusă din aplicația desktop și cea reală de funcționare, prezentă în acel moment pe echipamentul monitorizat.

Configurația locală a componentei este distribuită în trei fișiere cu roluri distincte, ce rețin în permanență informații legate de starea și funcționalitatea aplicației la nivelul dispozitivului. Primul fișier, „*mymodule.conf*”, localizat în „*/etc/*”, reprezintă fișierul principal de configurare, ce conține informații necesare comunicării cu componenta centrală, precum și informațiile de funcționare: modul curent activ și locația listei de conexiuni permise. Cel de al doilea fișier, „*mymodule_id*”, localizat în același director, stochează identificatorul unic al echipamentului.

Un al treilea fișier reține lista a conexiunilor permise, fișier a cărui locație și nume este specificată în „*mymodule.conf*”. Această flexibilitate permite administratorului să modifice și să plaseze informațiile în orice director al sistemului.

Inițializarea *daemon*-ului, realizată prin metoda *init()*, constă în deschiderea fișierului special „*/dev/mymodule*” în mod citire-scriere, prin care se realizează întreaga comunicare ulterioară cu modulul *kernel*. Dacă deschiderea eșuează, inițializarea este considerată nereușită și procesul se întrerupe. În continuare, componenta verifică existența fișierului „*mymodule_id*”. Dacă fișierul există și conține un identificator valid, acesta este încărcat în memorie pentru a fi inclus în toate mesajele transmise serverului. Dacă fișierul nu există, acesta este creat gol, urmând ca identificatorul să fie primit și salvat la prima comunicare reușită cu componenta centrală.

Comunicarea cu modulul *kernel* se realizează exclusiv prin apeluri *ioctl* asupra descriptorului de fișier deschis către „*/dev/mymodule*”. Comenzile definite acoperă toate operațiile necesare: *IOCTL_SET_SERVER_INFO* transmite adresa IP și portul componentei centrale, *IOCTL_SET_STATE* actualizează modul de funcționare curent, *IOCTL_CLEAR_WHITELIST* șterge lista conexiunilor permise din memoria *kernel*ului, *IOCTL_ADD_WHITELIST* adaugă intrări individuale în listă, iar *IOCTL_GET_PACKETS* recuperează un grup de pachete raportate de modul de la ultima interogare.

Interacțiunea cu componenta centrală se realizează prin REST API și implicit HTTP, pentru care s-a utilizat biblioteca *libcurl*. Datele schimbate sunt serializate în format JSON cu ajutorul bibliotecii *nlohmann/json*. Alegerea utilizării protocolului HTTP față de un protocol

TCP brut simplifică integrarea cu componenta centrală și permite utilizarea unor puncte de legătură REST bine definite.

Arhitectura internă este organizată în jurul a trei fire de execuție cu responsabilități distincte, pornite iterativ în funcția *main*.

Primul fir de execuție, „*run_conf_loop*”, monitorizează fișierele de configurare și sincronizează modificările dintre modulul *kernel* și server. La fiecare 15 secunde, firul verifică marcajul temporal de accesare al fișierului *mymodule.conf* și al fișierului de conexiuni permise prin apeluri *stat()*. Dacă marcajul diferă față de cel reținut de la ultima procesare, fișierul este recitit și modificările sunt transmise către modulul *kernel*. Parcurgerea fișierului *mymodule.conf* este realizată linie cu linie, ignorând liniile goale și comentariile marcate cu caracterul „#”. Sunt extrase din acest fișier adresa IP și portul serverului, locația fișierului de conexiuni permise și modul de funcționare impus. Pentru a include o toleranță la variații de capitalizare a informațiilor, modulul implementează conversia fiecărei linii la litere mici înainte de comparare.

Citirea fișierului de conexiuni permise, al cărui format a fost conceput cu accent pe simplitate și ușurință de citire, implică mai întâi ștergerea completă a listei curente din *kernel* prin comanda *IOCTL_CLEAR_WHITELIST*, urmată de adăugarea fiecărei intrări valide prin *IOCTL_ADD_WHITELIST*. În fișierul ce conține lista conexiunilor, fiecare linie va cuprinde o adresă a cărei acces este permis, existând posibilitatea de a specifica în continuarea adreselor o listă de porturi specifice. Pentru a restricționa comunicarea unei adrese IP la un număr de porturi, adresa va fi continuată de porturile permise: *a)*; iar pentru a oferi abilitatea de comunicare pe orice port, lista porturilor va fi omisă, pe linie rămânând doar adresa dispozitivului extern: *b)*.

192.168.0.111: 80, 81, 1234	a)
192.168.0.111	b)

Tabelul 3 - Formatul liniilor fișierului de conexiuni permise

Dacă oricare dintre fișiere a fost modificat și recitit cu succes, conținutul actualizat este trimis serverului printr-un POST la *endpoint*-ul „*/config/user?pc_id=...*”, ce permite componentei centrale să mențină o copie actualizată a configurației echipamentului. Marcajele temporale salvate sunt actualizate doar în cazul unei transmiteri reușite, ce asigură că o eroare de rețea nu marchează fișierul ca procesat, ci determină o nouă încercare la iterația următoare.

Al doilea fir de execuție, „*run_packet_loop*”, colectează în mod continuu pachetele raportate de modulul *kernel* și le transmite componentei centrale. La intervale de 100 de milisecunde,

firul apelează *IOCTL_GET_PACKETS*, ce primește o structură „*packet_patch*” cu pachetele acumulate de la ultima interogare. Dacă grupul citit conține cel puțin un pachet, acestea sunt serializate într-un obiect JSON ce va conține identificatorul echipamentului și o listă cu detaliile fiecărui pachet. Aceste informații sunt trimise printr-un POST la *endpoint*-ul „*/packets*”, iar din răspunsul serverului este extras identificatorul unic al echipamentului, care este persistat în fișierul „*mymodule_id*” în cazul în care acesta lipsește. Această logică acoperă cazul primei rulări, când identificatorul nu este încă cunoscut local și este atribuit de server la recepționarea primelor pachete. Componenta implementează un mecanism de reîncercare pentru pachetele ce nu au putut fi trimise din cauza unor erori de rețea: acestea sunt stocate într-o coadă de tip *std::deque*, cu un număr maxim de 1000 de elemente. La următoarea trimitere, pachetele din coadă sunt incluse în conținutul listei noi, ce urmează să fie trimisă. Prin aceasta se asigură că informațiile referitoare la traficul produs nu sunt pierdute în cazul unor întreruperi temporare ale conectivității.

Cel de al treilea fir de execuție, „*run_request_to_change_config_loop*”, implementează mecanismul prin care *daemonul* preia modificările de configurare inițiate de administrator. Similar transmiterii de informații descriptive, programul interoghează periodic serverul pentru a verifica apariția unor modificări produse de administrator, caz în care se activează secvența de actualizare de date. La fiecare 10 secunde, dacă identificatorul echipamentului este cunoscut, firul efectuează un apel GET la *endpoint*-ul „*/config/user?pc_id=...*”. Dacă răspunsul conține câmpul „*config_file*” sau „*allowed_file*”, noul conținut este scris pe disc prin utilizarea unui mecanism de înlocuire atomică în trei pași: conținutul nou este scris într-un fișier temporar cu extensia „*.tmp*”, fișierul curent este redenumit cu extensia „*.old*” ca și copie de rezervă, iar fișierul temporar este redenumit pentru a doua oară la forma finală. Dacă oricare dintre acești pași eșuează, copia de rezervă este restaurată. Această metodă asigură toleranța la erori și faptul că fișierul de configurare rămâne întotdeauna într-o stare validă. Modificările scrise pe disc sunt detectate automat de funcția „*run_config_loop*” la următoarea iterație a sa, prin verificarea marcajului temporal de modificare, ce va transmite informația modulului *kernel*, completând astfel ciclul de propagare a configurației de la interfața de administrare până la componenta de filtrare din spațiul *kernel*.

Oprirea controlată a componentei *daemon* este asigurată prin variabila booleană globală „*running*”, setată inițial pe valoarea *false*. Funcția „*interruptable_sleep*” utilizată în firele de execuție utilizate, verifică această variabilă la fiecare secundă, metodă ce permite firelor să iasă din starea de așteptare și să își termine execuția în mod sigur, fără a aștepta expirarea întregului interval configurat.

Componenta centrală

Componenta centrală reprezintă serverul aplicației și are rolul de a fi punctul principal de coordonare și comunicare între toate echipamentele ce sunt monitorizate și componenta de administrare. Aceasta este implementată în limbajul de programare *Python*, în care s-a folosit pentru crearea API-urilor *framework*-ul FastAPI și Uvicorn, prin care se expune o interfață REST utilizată atât de *daemonii* echipamentelor monitorizate, cât și de componenta de administrare.

Alegerea *framework*-ului FastAPI ca și soluție de dezvoltare a fost motivată de mai multe caracteristici relevante pentru acest context. FastAPI oferă suport nativ pentru programare asincronă prin utilizarea bibliotecii *asyncio*, ce permite gestionarea eficientă a unui număr mare de conexiuni simultane fără a bloca firul de execuție. Validarea automată a datelor de intrare și ieșire este realizată prin intermediul bibliotecii *Pydantic*, ce elimină necesitatea implementării manuale de verificări a tipului de date și al structurilor, reducând riscul erorilor de procesare a datelor. Uvicorn, serverul ASGI (*Asynchronous Server Gateway Interface*) este utilizat pentru performanță ridicată și latență minimală.

Structurile de date utilizate în cadrul componentei centrale sunt create prin utilizarea claselor *Pydantic*, care asigură validarea și serializarea automată a datelor. Modelul *PacketData* descrie un pachet de rețea raportat de echipamentul monitorizat, incluzând adresele IP, porturile și MAC sursă cât și destinație, protocolul TTL-ul, lungimea pachetului, flag-urile TCP în cazul în care există, interfața de rețea și modul de funcționare activ la momentul interceptării. Modelul *PacketsRequest* grupează o listă de pachete împreună cu identificatorul echipamentului sursă. Modelul *FilesConfig* stochează configurația unui echipament, care include conținutul fișierelor *mymodule.conf* și al fișierului de conexiuni permise, cât și o iconiță reprezentativă echipamentului și numele echipamentului introduse de administrator pentru identificare ușoară și marcajul de timp al ultimei modificări de configurare. Modelul *TopologyEntry* și *ConnectionStats* descriu structura datelor de topologie, în care se stochează pentru fiecare echipament monitorizat statistici detaliate despre fiecare sursă de conexiune observată.

Un aspect central al arhitecturii serverului este utilizarea unui nivel de *cache* în memorie RAM, reprezentat de un dicționar intern *cache_dict*. Fiecare intrare din acest dicționar, indexată după identificatorul unic al echipamentului, conține trei elemente: instanța curentă a configurației echipamentului de tip *FilesConfig*, un câmp boolean *update* care semnalizează

existența unor modificări de configurare ce trebuie propagate *daemon*-ului și o coadă *deque* cu pachetele recente raportate de echipament. Această strategie de *cache* elimină necesitatea de accesare a bazei de date în mod repetat pentru fiecare cerere de citire a configurației sau a rapoartelor recente, operații ce apar în mod repetat în aplicația implementată. Baza de date este utilizată ca și metodă de a persista informațiile pe termen lung, verificată la pornire pentru construirea stării și scrisă la oprire pentru salvarea pe disc a modificărilor acumulate în memorie pe durata rulării.

Un alt dicționar de tip *cache* aflat în memorie, *connection_summary_map*, stochează datele de topologie extrase din pachetele raportate. Acesta este populat în timp real la recepția fiecărui grup de pachete și citit direct din memorie în momentul solicitării topologiei, fără a implica accesul la baza de date.

Inițializarea și oprirea aplicației sunt gestionate prin mecanismul *lifespan* oferit de *FastAPI*, care permite definirea unor operații ce se execută o singură dată la pornire și la oprire. La pornire, componenta centrală deschide conexiunea cu baza de date *SQLite*. În cazul în care acesta nu este găsită, baza de date este generată automat. Dacă inițializarea acesteia eșuează, aplicația este întreruptă. La oprire, conținutul componentei *cache_dict* este parcurs iar configurația fiecărui echipament este persistată în baza de date, proces prin care se asigură că modificările efectuate în timpul rulării nu sunt pierdute la oprire.

Componenta centrală expune șapte *endpoint*-uri REST cu responsabilități distincte, grupate în trei categorii funcționale: recepționarea pachetelor, gestionarea configurațiilor și interogarea datelor.

- *GET /config/user* este apelat periodic de *daemoni* pentru a verifica existența unor modificări de configurare. Logica de răspuns implementează un mecanism eficient de *pull*: actualizarea configurării se face după starea curentă a variabilei *update* din interiorul componentei destinată echipamentului interogat, care se află în *cache_dict*. În cazul în care acesta este în starea de *True*, configurația curentă este returnată, iar variabila ce indică existența unor modificări este resetată. În cazul în care acesta este *False*, serverul returnează un răspuns compact ce indică absența modificărilor, evitând traficul inutil de date. Dacă echipamentul nu are un câmp în *cache*, serverul consultă baza de date și, dacă se găsește o configurație persistentă, o încarcă în *cache* înainte de a răspunde.
- *POST /config/user* este utilizat de către *daemoni* pentru a notifica serverul despre modificări ce au apărut local în fișierele de configurare. La recepție, serverul

actualizează câmpurile corespunzătoare din *cache_dict*, fără a scrie imediat în baza de date. Marcajele temporale de modificare al configurației sunt actualizate pentru câmpurile care au primit valori noi.

- *GET /config/admin* servește componenta de administrare, și returnează fie configurația unui echipament specific, fie lista configurațiilor tuturor echipamentelor cunoscute. Când cererea nu specifică niciun identificator, serverul construiește o listă prin combinarea echipamentelor prezente în *cache_dict* cu cele existente exclusiv în baza de date, completând câmpurile lipsă din *cache*. Această abordare aleasă asigură faptul că administratorul vede atât echipamentele active curent, cât și echipamentele care au comunicat anterior cu serverul, dar nu sunt momentan conectate (sau nu și au mai modificat configurația de la ultima repornire al serverului).
- *POST /config/admin* este utilizat de asemenea de componenta de administrare pentru modificarea configurației unui echipament. Spre deosebire de *endpoint*-ul destinat componentei de intermediere, acesta setează variabila *update* pe starea *True* atunci când câmpurile de configurare *config_file* sau *allowed_file* sunt modificate, eveniment ce declanșează propagarea modificărilor către *daemon* la următoarea interogare de configurare a sa. Câmpurile destinate utilizării strict în componenta administrativă pot fi de asemenea actualizate prin acest punct de comunicare, fără a declanșa actualizarea configurației echipamentului.
- *GET /logs* oferă acces la istoricul evenimentelor de securitate, cu suport pentru filtrare în funcție de identificatorul echipamentului și numărului de rezultate cerute. Dacă echipamentul este prezent în *cache*, *log*-urile sunt returnate direct din memorie, fără acces la baza de date. În caz contrar, sunt returnate *log*-urile din coada globală stocată pe disc, filtrare după identificator. Această strategie aleasă prioritizează viteza de răspuns pentru echipamentele active, recurgând din baza de date doar pentru interogări istorice.
- *GET /topology* și *GET /topology/reset* oferă acces la datele de topologie acumulate și respectiv resetarea acestora. Datele de topologie sunt returnate direct din *connection_summary_map*, structură din memorie ce se actualizează la recepția fiecărui grup nou de pachete.

Funcția *topology_extraction* este responsabilă de construirea progresivă a hărții conexiunilor din rețea, pe baza pachetelor raportate. Pentru fiecare echipament monitorizat, identificat prin

pc_id, este menținută o instanță *TopologyEntry* care conține un set cu toate adresele IP de destinație observate, reprezentând adresele folosite ale echipamentului în rețea, și un dicționar indexat după adresa IP sursă a fiecărui inițiator de conexiune. Pentru fiecare sursă se acumulează informațiile de interes ale pachetelor transmise.

Serverul conține o clasă *Database* care encapsulează toate operațiile de acces la baza de date *SQLite* prin utilizarea de conexiuni independente pentru fiecare operație, ce asigură siguranța utilizării pe fire de execuție multiple. Această abordare, comparativ cu utilizarea unei conexiuni globale, elimină probleme de concurență specifice *framework*-ului *SQLite* în scenarii de utilizare în care sunt utilizate multiple fire de execuție, soluție ce prezintă costuri minore de procesare: deschiderea conexiunii de mai multe ori.

La inițializare, baza de date este configurată în modul *WAL (Write-Ahead Logging)* activat prin comanda *PRAGMA journal_mode=WAL*. Modul *WAL* permite citiri concurente în timp ce o scriere este în curs, metodă ce îmbunătățește semnificativ performanța în scenariul în care pachetele sunt inserate frecvent în timp ce componenta de administrare interoghează simultan informațiile.

Schema bazei de date include indecși pe coloanele utilizate pentru interogări: marcajul temporal și identificatorul echipamentului, pentru filtrarea și sortarea *log*-urilor prezente[Figura 3 - Configurare bază de date].

```
CREATE TABLE IF NOT EXISTS connections (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  mode INTEGER,  
  pc_id TEXT NOT NULL,  
  ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  src_ip TEXT NOT NULL,  
  dst_ip TEXT NOT NULL,  
  src_port INTEGER,  
  dst_port INTEGER,  
  protocol INTEGER,  
  ttl INTEGER,  
  packet_len INTEGER,  
  iface TEXT,  
  src_mac TEXT,  
  dst_mac TEXT,  
  tcp_flags INTEGER  
);  
  
CREATE TABLE IF NOT EXISTS pc_configs (  
  pc_id TEXT PRIMARY KEY,  
  config_file TEXT,  
  allowed_file TEXT,  
  icon BLOB,  
  name TEXT DEFAULT "",  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Figura 3 - Configurare bază de date

Componenta de administrare

Componenta de administrare este reprezentată de aplicația desktop destinată administratorului rețelei, implementată în limbajul C++. Aceasta este construită pe un ansamblu de biblioteci specializate care acoperă randarea grafică, comunicarea cu componenta centrală și procesarea datelor vizuale.

Fundația aplicației o reprezintă biblioteca *Dear ImGui*, o bibliotecă grafică *open-source* de tip *immediate mode GUI*. În acest tip de bibliotecă, spre deosebire de conceptul *retained mode GUI*, specifică majorității *framework*-urilor de interfață grafică, în care starea interfeței este menținută implicit de bibliotecă și în care actualizarea se face prin funcții *callback*, paradigma *immediate mode* construiește întreaga interfață la fiecare cadru, pe baza stării curente a aplicației. Fiecare element grafic este creat, afișat și interogată în aceeași componentă, returnând informații despre interacțiunea cu utilizatorul. Această abordare ajută la simplificarea semnificativă a nevoii de sincronizare de date deoarece elimină complet necesitatea mecanismelor de *binding* sau a componentelor de tip *observer*. Aceste avantaje sunt accentuate de contextul aplicației, în care datele de afișare se actualizează frecvent pe baza evenimentelor primite de la server.

Dear ImGui nu cuprinde în mod direct logica de gestionare a unei ferestre sau un context de randare, aceste sarcini fiind alocate unor componente specializate. Aplicația folosește *GLFW* pentru crearea și gestionarea ferestrei sistemului și pentru procesarea evenimentelor, și *OpenGL3* ca și *backend* de randare, prin care *ImGui* își translatează comenzile de desenare în apeluri grafice native. Integrarea este realizată prin modulele „*imgui_impl_glfw*” și „*imgui_impl_opengl3*”, care asigură inițializarea corectă și sincronizarea dintre cele trei biblioteci.

Comunicarea cu componenta centrală este realizată prin biblioteca *libcurl*, utilizată pentru efectuarea tuturor cererilor *HTTP* către *endpoint*-urile *REST* ale serverului. *Libcurl* oferă un API complet pentru cereri sincrone cu suport pentru expirarea conexiunilor și transferului de date, configurate pentru reapelare la două respectiv cinci secunde pentru a preveni blocarea firului principal de execuție în cazul indisponibilității serverului. Datele primite în format JSON de la server sunt interpretate cu ajutorul bibliotecii „*nlohmann/json*”, utilizată de asemenea și în componenta de intermediere, pentru a asigura o consistență a soluțiilor folosite în întreg sistemul.

Încărcarea și procesarea imaginilor este realizată prin biblioteca *header-only* „*stb_image*”. Aceasta suportă decodarea imaginilor în formate uzuale precum *PNG* și *JPEG*. Imaginile sunt decodate în memorie și încărcate ca și texturi OpenGL prin funcțiile „*load_image_as_texture*” pentru fișiere de pe disc și „*load_texture_from_memory*” pentru format *base64*. Gestionarea texturilor *OpenGL* implică alocarea unui identificador numeric prin *glGenTextures*, configurarea parametrilor de filtrare și încărcarea datelor pixelilor prin *glTexImage2D*, urmată de eliberarea explicită a texturilor vechi prin *glDeleteTextures* la fiecare actualizare, pentru a preveni scurgerile de memorie video.

Arhitectura aplicației

Aplicația este structurată în trei clase cu responsabilități individuale, ce corespund celor trei pagini ale interfeței:

- *DashBoard* – pentru pagina principală
- *HostDetail* – pentru pagina individuală unui echipament
- *Topology* – pentru vizualizarea topologiei rețelei

Navigarea între pagini este gestionată printr-o variabilă globală *page* de tip enumerație, ce se modifică la apăsarea butoanelor de navigare sau la interacțiunea cu elementele dedicate din interiorul paginilor.

Bucula principală a aplicației, urmează structura standard a aplicațiilor *ImGui*: procesarea evenimentelor *GLFW*, inițierea unui nou cadru *ImGui*, randarea conținutului paginii active prin apelul metodei *draw* a clasei corespunzătoare, finalizarea randării și afișarea rezultatului. Rata de cadre este limitată la 45 de cadre pe secundă deoarece se dorește evitarea consumului inutil al procesorului între perioade de inactivitate mari.

Fiecare dintre cele trei clase principale implementează un interval de reîmprospătare propriu, definit ca și constantă statică *REFRESH_INTERVAL*. Clasa *Dashboard* și clasa *HostDetail* folosesc un interval de cinci secunde, în timp ce clasa *Topology* utilizează un interval de șapte secunde, datorită volumului mai mare de date preluate și calculului care necesită o putere de procesare mai mare. La fiecare cadru, funcția *draw* a paginii active compară timpul curent cu cel al ultimei reîmprospătări și, dacă intervalul a expirat, declanșează o nouă interogare a componentei centrale, independent de randare.

Pagina principală – Dashboard

Din punct de vedere funcțional, clasa *Dashboard* menține două structuri de date principale: *pc_map*, un pointer către un *map* indexat după identificadorul echipamentului, populat prin

funcția *get_pc_info*, și *msg_history_vector*, populat prin funcția *get_message_history*, ce conține cele mai recente 20 de evenimente a tuturor echipamentelor. La fiecare reîmprospătare, cele două structuri sunt reconstruite integral din răspunsul componentei centrale, fără a păstra o stare incrementală locală.

Funcționalitatea de adăugare incrementală la nivel global este implementată ca o mașină de stări simplă, controlată de variabila *training_activate*. La activare, aplicația parcurge întregul *pc_map* și, pentru fiecare echipament, apelează funcția *config_change_state* asupra conținutului curent al fișierului de configurare pentru a recrea fișierul de configurații cu modul de funcționare modificat în MONITORING. Modul anterior al fiecărui echipament este reținut într-un dicționar *training_pc_mode_before*, indexat după *pc_id*, iar noua configurație este transmisă componentei centrale pentru actualizarea echipamentului prin *send_config_via_api*. De asemenea, la același pas, este preluată starea curentă a topologiei, stocată în variabila *training_topology_map*, care va servi drept referință pentru sesiunea de antrenare.

La dezactivare, aplicația parcurge *training_topology_map* și agregă pentru fiecare echipament toate porturile sursă (*ports_out*) observabile pentru fiecare adresă IP inițitoare de conexiune într-o structură intermediară *training_allowed_new_elements*, de tip *map<string,set<int>>*. Ulterior, pentru fiecare echipament prezent în dicționarul *training_pc_mode_before*, este reconstruită configurația cu modul de funcționare restaurat la valoarea salvată, iar fișierul de conexiuni permise este regenerat prin funcția *merge_allowed_with_ports_map*, care combină intrările existente cu porturile noi acumulate, păstrând totodată intrările ce permit comunicarea pe orice port nemodificate. Cele două fișiere actualizate sunt transmise echipamentului printr-un singur apel, prin funcția *send_config_via_api*.

Pagina individuală a echipamentului – HostDetail

Clasa *HostDetail* tratează configurația unui echipament și gestionează stările aplicației prin două seturi paralele de date: starea curentă a echipamentului, reprezentată de structura *PCInfo* populată din răspunsul serverului, și starea curentă a paginii, editată local, distribuită într-un set de liste de caractere (*server_ip_buf*, *server_port_buf*, *allowed_file_path_buf*, *name_given_buf*) și o variabilă numerică reprezentativă modului curent de funcționare *changed_config_mode*. Fiecare grup logic de informații este asociat unui indicator boolean propriu: *changed_config_checker*, *changed_allowed_file_checker*, *changed_general_checker*; care marchează existența unor modificări nesincronizate.

Metoda *update_new_frame* realizează sincronizarea cu serverul, dar evită suprascrierea modificărilor locale neaplicate; reîncărcarea completă a variabilelor are loc doar atunci când câmpul *updated_at* primit din partea serverului diferă de cel reținut local sau când reîmprospătarea este forțată din aplicație în mod indirect, la schimbarea echipamentului selectat. Parcurgerea conținutului brut al fișierelor de configurare este realizată prin funcția *extract_config_info*, care extrage adresa IP, portul și calea fișierului de conexiuni permise pe baza prefixelor liniilor, și prin funcția *extract_current_state*, care identifică modul de funcționare prin căutarea textuală, insensibilă la capitalizare, a uneia dintre cele patru valori posibile. Lista de conexiuni permise, primită ca text, este descompusă linie cu linie în vectorul *allowed_lines*, care constituie sursa de adevăr pentru operațiile de editare ulterioare.

La salvarea modificărilor făcute din aplicație, metoda *create_config_info_string* realizează operația inversă, reconstruind textul fișierului *mymodule.conf* din valorile curente ale *buffer*-elor și din indexul modului selectat în lista ce conține modurile de funcționare *MODES*, iar vectorul *allowed_lines* este searializat înapoi linie cu linie. Rezultatul este transmis prin funcția *send_config_via_api*, care construiește un obiect JSON ce conține doar câmpurile modificate și efectuează o cerere POST către *endpoint*-ul */config/admin*.

Funcționalitatea de antrenare la nivelul echipamentului utilizează aceeași logică precum varianta globală, dar este limitată la o singură sesiune activă simultan, identificată prin variabila *training_active_id*. La activare, configurația curentă este salvată în variabila *config_before_training*, echipamentului i se schimbă modul de funcționare în *MONITORING* prin funcția *config_change_state* și este inițiată preluarea topologiei specifice acestui echipament prin funcția *get_topology*, stocată în variabila *msg_struct_info_map*. La dezactivare, funcția *merge_allowed_with_topology* integrează porturile observate în sesiune direct în *allowed_lines*, iar modul de funcționare original este restaurat prin reaplicarea funcției *config_change_state* asupra configurației salvate.

Pagina topologiei rețelei – Topology

Construcția grafului de topologie combină, de asemenea, două surse de date: *get_pc_info*, care oferă lista echipamentelor monitorizate și fișierele lor de conexiuni permise, și *get_topology*, care, pentru fiecare echipament, returnează mulțimea adreselor IP cunoscute ale acestuia: *topology_ip*, și statistici per adresă sursă a conexiunilor primite: *connection_dict*.

Pe baza acestor date sunt construite mai multe structuri intermediare:

- *ip_to_pc_id* reprezintă o mapare inversă, de la fiecare adresă IP cunoscută a unui echipament către identificatorul acestuia, ce permite recunoașterea conexiunilor dintre echipamentele monitorizate
- *allowed_ips_map* construit pentru fiecare echipament prin funcția *extract_allowed_ips*, ce transformă textul fișierului de conexiuni permise într-o structură *std::map<std::string, std::set<int>>*, unde o mulțime de porturi vidă semnifică permisiune nerestricționată, iar mulțime cu elemente semnifică restricționarea comunicării pe acele porturi.
- *connections_map* agregă, pentru fiecare nod sursă (fie identificatorul unui echipament monitorizat, fie adresa IP externă), mulțimea echipamentelor către care a inițiat conexiuni.

Determinarea conexiunilor neautorizate, evidențiate vizual în pagina de topologie, este realizată printr-un algoritm care, pentru fiecare echipament receptor și fiecare adresă sursă din *connection_dict*, exclude mai întâi adresele care apar în propriul *topology_ip* al receptorului – acesta reprezentând traficul intern al echipamentului către sine, nu o conexiune externă. Pentru adresele rămase, se determină dacă sursa corespunde unui alt echipament monitorizat, prin maparea *ip_to_pc_id*, sau a unui dispozitiv extern. La întâlnirea celor două cazuri, adresa este verificată față de *allowed_ips_map* al receptorului: absența acesteia, sau prezența ei cu o mulțime de porturi care nu se suprapune listei de porturi permise marchează conexiunea ca și nepermisă, prin funcția *addConnectionBlocked*, ce adaugă conexiunea într-o structură consultată anterior la randare pentru colorarea muchiilor.

Poziționarea nodurilor este delegată utilitarului *open-source* extern **Graphviz** [50], versiunea 2.43.0, prin algoritmul „*fdp*” (*Force-Directed Placement*): mulțimea nodurilor și relațiile din *connection_map* sunt scrise într-un fișier temporar în format DOT. *Graphviz* este invocat ca proces extern, iar fișierul de ieșire în format *plain* este parcurs pentru a extrage coordonatele calculate ale fiecărui nod [Figura 4 - Generare graf]. Poziționarea este recalculată o singură dată per sesiune de vizualizare – fie la prima randare, fie după o resetare explicită – și este normalizată la o plajă fixă de coordonate, pentru a asigura o distribuție consistentă pe ecran, indiferent de numărul de noduri.

Randarea propriu-zisă este realizată direct prin canvasul *ImGui* prin primitiva *ImDrawList*. Pentru fiecare nod sunt calculate punctele de conexiune (pinii) către nodurile vecine prin funcția *get_closest_edge*, care determină, dintre cele patru puncte mediane ale marginilor unui

```

// Preluare locația fișierelor temporale
std::string temp_dir;
#ifdef _WIN32
char temp_path[MAX_PATH];
GetTempPathA(MAX_PATH, temp_path);
temp_dir = temp_path;
#elif __linux__
temp_dir = "/tmp/";
#endif

std::string dot_file_path = temp_dir + "graph.dot";
std::string txt_file_path = temp_dir + "graph.txt";

// Generare dot_file
std::string dot = "graph G {\n";
dot += "node [shape=box];\n";
dot += "graph [overlap=false, sep=\"1\", K=0.1];\n";
...

#ifdef _WIN32
std::string cmd = "fdp -Tplain -Gstart=2 \"" + dot_file_path + "\" -o \"" + txt_file_path + "\"";
#else
std::string cmd = "fdp -Tplain -Gstart=2 " + dot_file_path + " -o " + txt_file_path + " 2>/dev/null";
#endif
system(cmd.c_str());

```

Figura 4 - Generare graf

dreptunghi, cel mai apropiat de centrul nodului țintă Prin aceasta se asigură că muchiile pornesc și se termină pe marginea vizibilă a nodurilor, nu din centrul acestora. Muchiile sunt randate ca și curbe *Bezier* cubice, cu punctele de control deplasate perpendicular pe direcția muchiei, pentru a evita suprapunerea liniilor în cazul conexiunilor multiple între aceleași noduri.

Interacțiunea utilizatorului cu canvasul este gestionată prin funcția *calculate_zoom_and_drag*: roțița *mouse*-ului modifică factorul *topology_zoom*, limitat la intervalul 0.3 – 3.0, iar la fiecare modificare poziția de pan *topology_pan* este recalculată astfel încât punctul aflat sub cursor să rămână fix pe ecran. Tragerea cu butonul stâng acumulează deplasarea *mouse*-ului direct în *topology_pan*. Conversia finală din coordonatele de „lume” în coordonate de ecran este realizată prin funcția *to_screen*, care aplică transformarea:

$$poziție_ecran = poziție_lume \times zoom + pan$$

Fluxul de funcționare al sistemului

Înainte de a discuta despre interfață și interacțiunile disponibile administratorului, este utilă o recapitulare a modului în care cele patru componente cooperează pe durata funcționării sistemului, comunicare ilustrată de diagrama de compunere [Figura 5 - Diagramă de

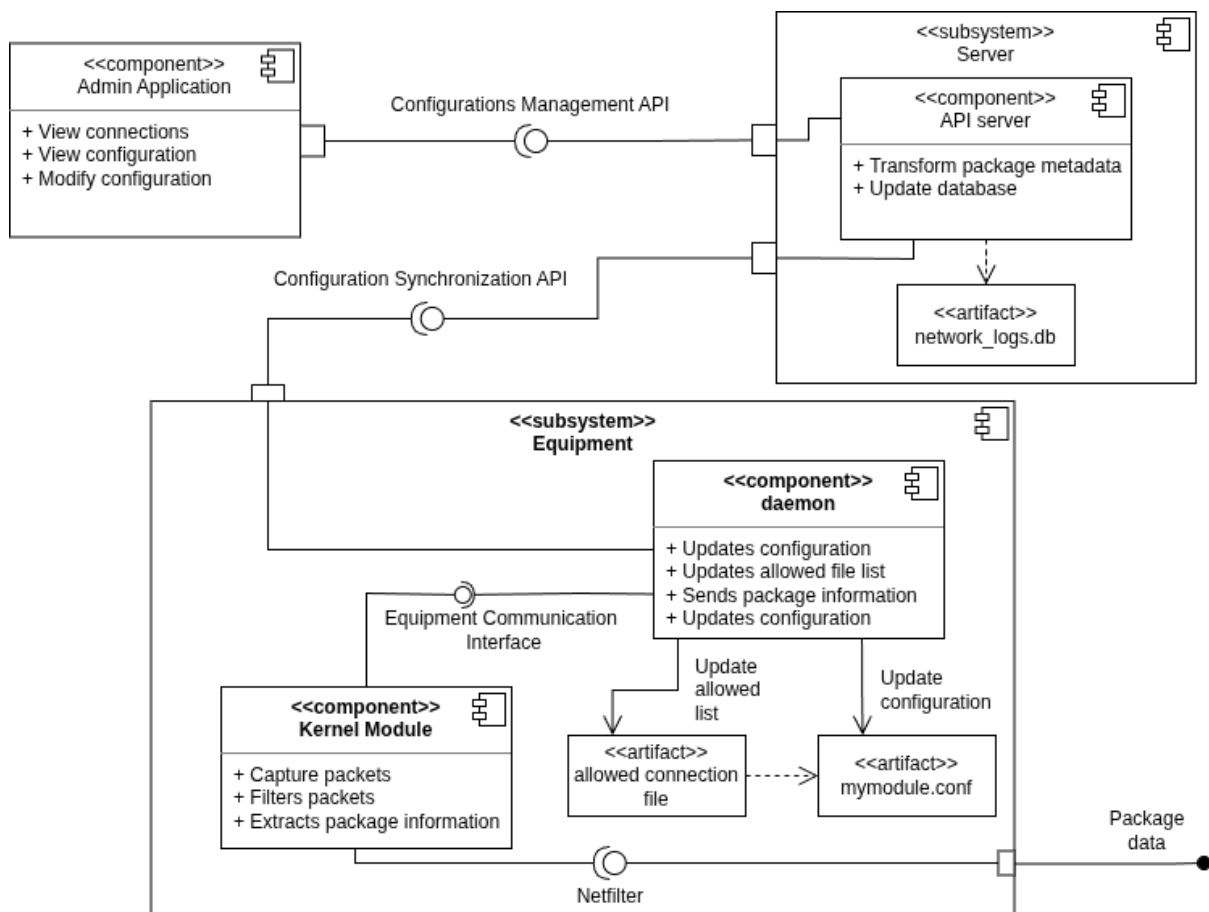


Figura 5 - Diagramă de componente

componente] și cele două diagrame secvențiale: a traseului parcurs de informațiile asociate pachetelor interceptate de *componenta de filtrare* [Figura 6 - Diagramă secvențială - informații pachete] și respectiv a fluxului de actualizare și propagare a configurației echipamentelor [Figura 7 - Diagramă secvențială – configurații].

La inițializare, componenta de filtrare înregistrează *hook-ul Netfilter* în stiva de rețea a sistemului de operare [Figura 6 - Diagramă secvențială - informații pachete], pas din care orice pachet destinat echipamentului va trebui să treacă prin funcția *packet_hook* înainte de a ajunge la procesele locale. În paralel, componenta de intermediere citește fișierele de configurare locale și, dacă acestea conțin informații mai recente decât cele cunoscute de server, le transmite componentei centrale [Figura 7 - Diagramă secvențială – configurații], în timp ce componenta de administrare interoghează la rândul ei serverul pentru a obține și afișa configurația curentă a fiecărui echipament.

Pe durata funcționării normale (*Runtime Monitoring*), cele două fluxuri devin ciclice. La recepția unui pachet, componenta de filtrare verifică sursa acestuia față de lista conexiunilor permise și, în funcție de modul curent, decide acceptarea sau refuzul pachetului (opțiunile

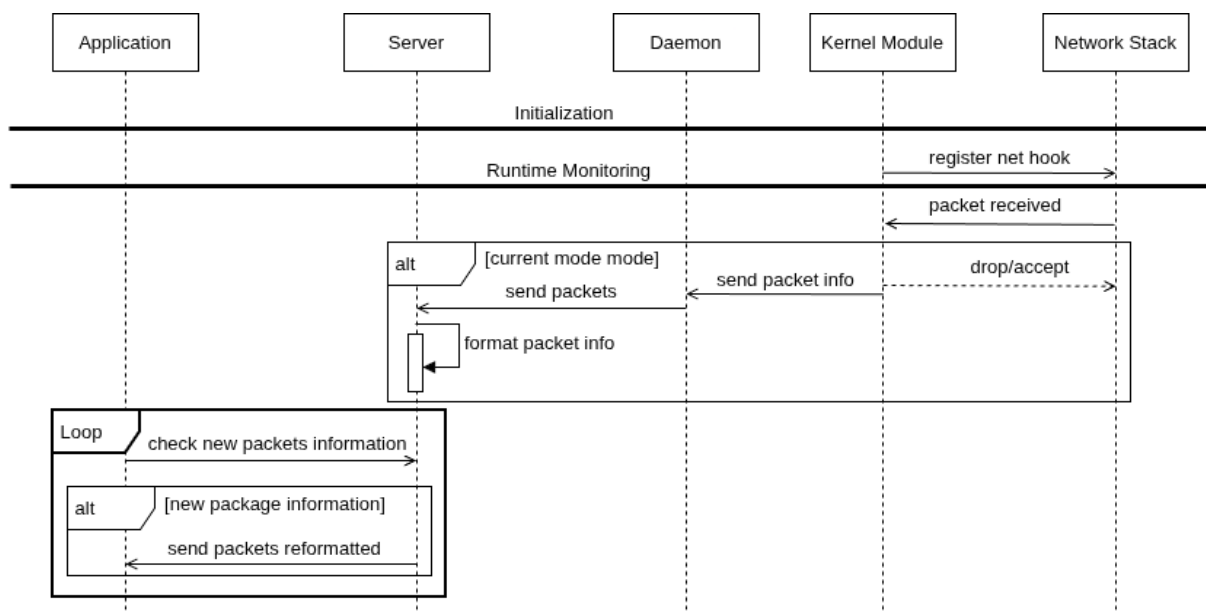


Figura 6 - Diagramă secvențială - informații pachete

drop/accept), informație transmisă direct stivei de rețea, extrăgând simultan informațiile relevante din pachete și transmițând aceste date componentei de intermediere [Figura 6 - Diagramă secvențială - informații pachete]. Acesta acumulează informațiile primite și, periodic, le reformatează și le trimite componentei centrale, care le salvează în baza de date și le pune la dispoziția componentei de administrare.

Fluxul de configurare urmează un model simetric [Figura 7 - Diagramă secvențială – configurații]. Atunci când administratorul modifică, din aplicația desktop, modul de funcționare sau lista conexiunilor permise ale unui echipament, modificarea este transmisă componentei centrale. Componenta de intermediere, care interoghează periodic serverul, detectează existența unei configurații noi, o transmite componentei de filtrare pentru aplicarea imediată a noilor reguli și o persistă local, pe disc, pentru a asigura continuitatea configurației la o eventuală repornire a echipamentului.

Această buclă în care evenimentele observate de componenta de filtrare ajung la administrator prin componenta centrală, iar deciziile administratorului ajung înapoi la componenta de filtrare pe același traseu, reprezintă mecanismul de bază prin care sistemul oferă vizibilitate și control asupra traficului din rețeaua industrială.

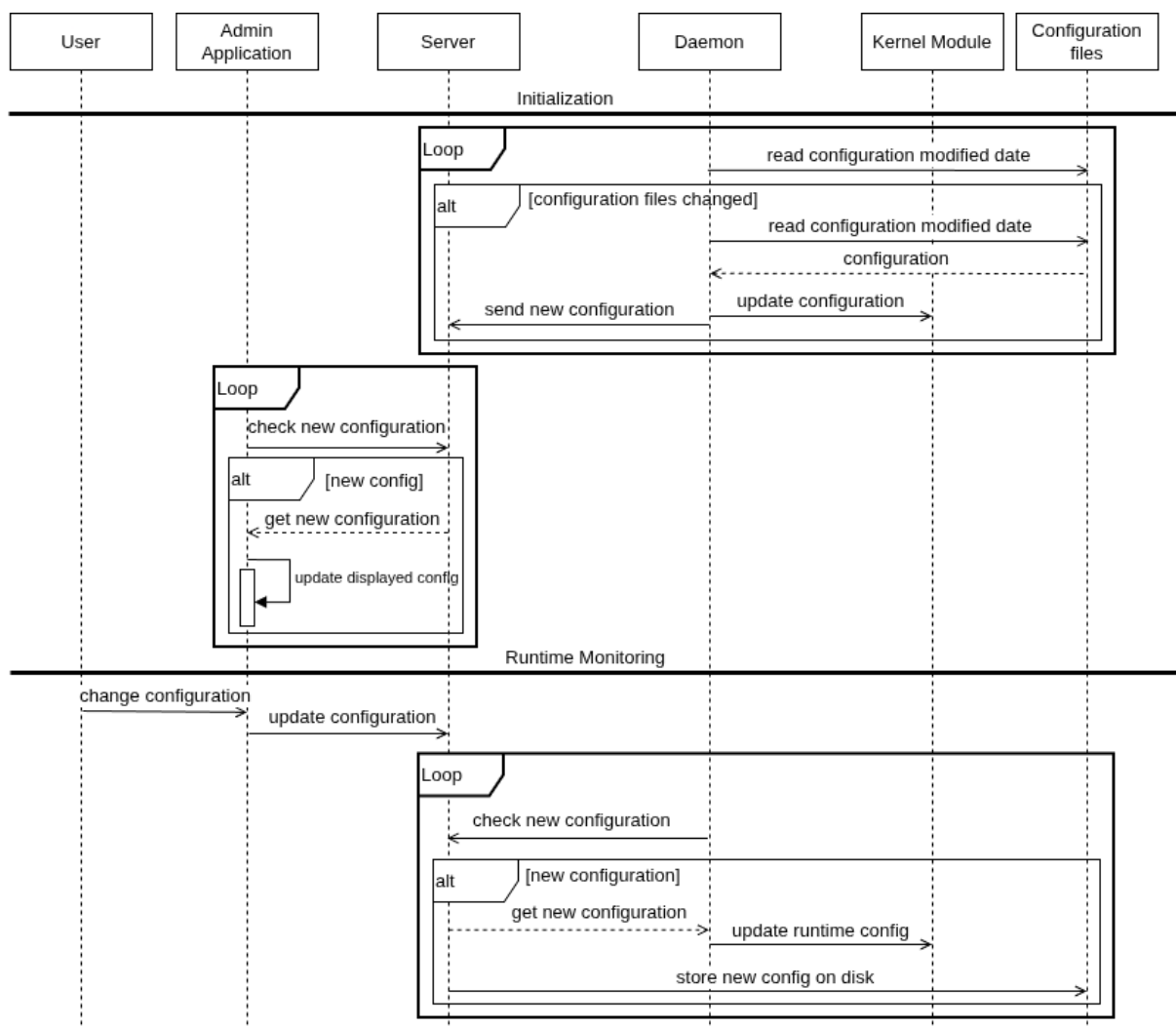


Figura 7 - Diagramă secvențială – configurații

Experiența utilizatorului

Pentru asigurarea rulării corecte a aplicației, se recomandă utilizarea pe echipament a unei distribuții *Linux* (pentru testare s-a folosit *Debian*) cu versiunea de *kernel* 6.8.0-51, dependență ce provine din interfețele interne ale *kernel*-ului utilizate de componenta de filtrare, care pot varia între versiuni.

Configurarea unui echipament nou pentru monitorizare presupune parcurgerea a patru pași:

1. **Crearea fișierelor de configurare:** Aplicația caută în memoria echipamentului fișierul „*mymodule.conf*”, în locația „*/etc/*”. În acest fișier [Figura 8 - *mymodule.conf*] va fi definită adresa de IP locală a serverului căruia i se vor trimite informațiile referitoare la pachete, portul pe care acesta va trebui să în folosească în

comunicarea cu serverul, locația fișierului ce va fi utilizat în definirea conexiunilor permise și modul de funcționare a aplicației pe dispozitivul suport.

```
# Pentru a comenta o linie, se poate folosi
# caracterul „#”

# exemplu de adresă - 192.168.0.112
server IP <adresa ip a serverului>

# portul trebuie regăsit în valoare numerică
server port <port>

# se dorește locația globală a fișierului
allowed file <locația fișierului de definiții>

<mod>
# Modul de rulare este procesat fără diferențiere între
# majuscule și minuscule.
# se acceptă – listening, Listening, LISTENING, ListEnInG
```

Figura 8 - *mymodule.conf*

2. **Compilarea modulului kernel** se realizează prin comenzia „*make*”. Această comandă utilizează fișierul de configurare *Makefile* [Figura 9. *Makefile*] care utilizează arborele de surse al kernelului curent pentru a asigura compatibilitatea modulului rezultat cu versiunea instalată.

```
obj-m += kernel_module.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Figura 9. *Makefile*

3. **Adăugarea modulului în spațiul kernel** realizată prin rularea comenzii „*sudo insmod kernel_module.ko*” rulată cu privilegii elevate, care înregistrează *hook*-ul *Netfilter* și creează *character device*-ul */dev/mymodule*.
4. **Rularea daemonului** se face prin comanda „*sudo ./daemon*” rulată cu privilegii elevate pentru a permite accesul la *character device*-ul creat de componenta de filtrare.

După parcurgerea acestor pași, echipamentul devine vizibil în aplicația desktop și începe să raporteze periodic evenimentele către componenta centrală.

La pornire, aplicația desktop deschide o fereastră cu rezoluția 1400x900 de pixeli, intitulată *Network Inspection App*, cu fundal de culoare gri-albăstrui. În partea superioară a ferestrei se află un meniu de navigare format din două butoane – *Main page* și *Network Topology* – ce apar în permanență, indiferent de pagina curentă, ce permite astfel comutarea rapidă între cele trei secțiuni ale aplicației.

Pagina principală se deschide cu titlul *Network Monitoring System* [Figura 10 - Pagina principală]. Sub meniul de navigare se găsește butonul *Train*, prin care se începe procesul de adăugare automată a conexiunilor detectate, eveniment petrecut la nivelul tuturor echipamentelor din rețea. După comutare, butonul se schimbă în *Stop Training*, iar o nouă apăsare finalizează adăugarea și aplică automat regulile de acces nou-găsite.

Sub acest buton se află secțiunea *Connected Clients*, o listă derulabilă în care fiecare echipament monitorizat este reprezentat printr-un dreptunghi de 500x55 de pixeli, cu colțuri rotunjite și un contur de culoare gri deschis. Culoarea de fundal a fiecărui dreptunghi indică modul de funcționare curent al echipamentului:

1. Roșu – REACTIVE
2. Verde – LISTENING
3. Albastru - MONITORING
4. Gri – DISABLED

Pentru modurile MONITORING și REACTIVE, fundalul elementului pulsează – opacitatea culorii alternează la fiecare jumătate de secundă – atrăgând atenția administratorului asupra echipamentelor aflate într-o stare tranzitorie sau care raportează activitățile blocate. În partea stânga a fiecărui dreptunghi este afișată iconița echipamentului (dacă a fost adăugată una), în dreapta acestuia apar numele echipamentului (sau în absența unui nume, identificatorul său unic), și sub acesta, cu text gri, data și ora ultimei modificări. În colțul din dreapta dreptunghiului, modul curent de funcționare este afișat, și precedat de textul „Mode:”, scris cu aceeași culoare folosită pentru codificarea fundalului.

Trecerea cursorului peste un dreptunghi reprezentativ echipamentului schimbă forma acestuia într-o mână, semnalând faptul că acestea sunt componente interactive: un click stânga va deschide pagina individuală a echipamentului respectiv, iar un click dreapta deschide un mic meniu contextual, care afișează identificatorul echipamentului și, în momentul de față, o singură opțiune: *Edit Config*, ce conduce de asemenea către pagina individuală.

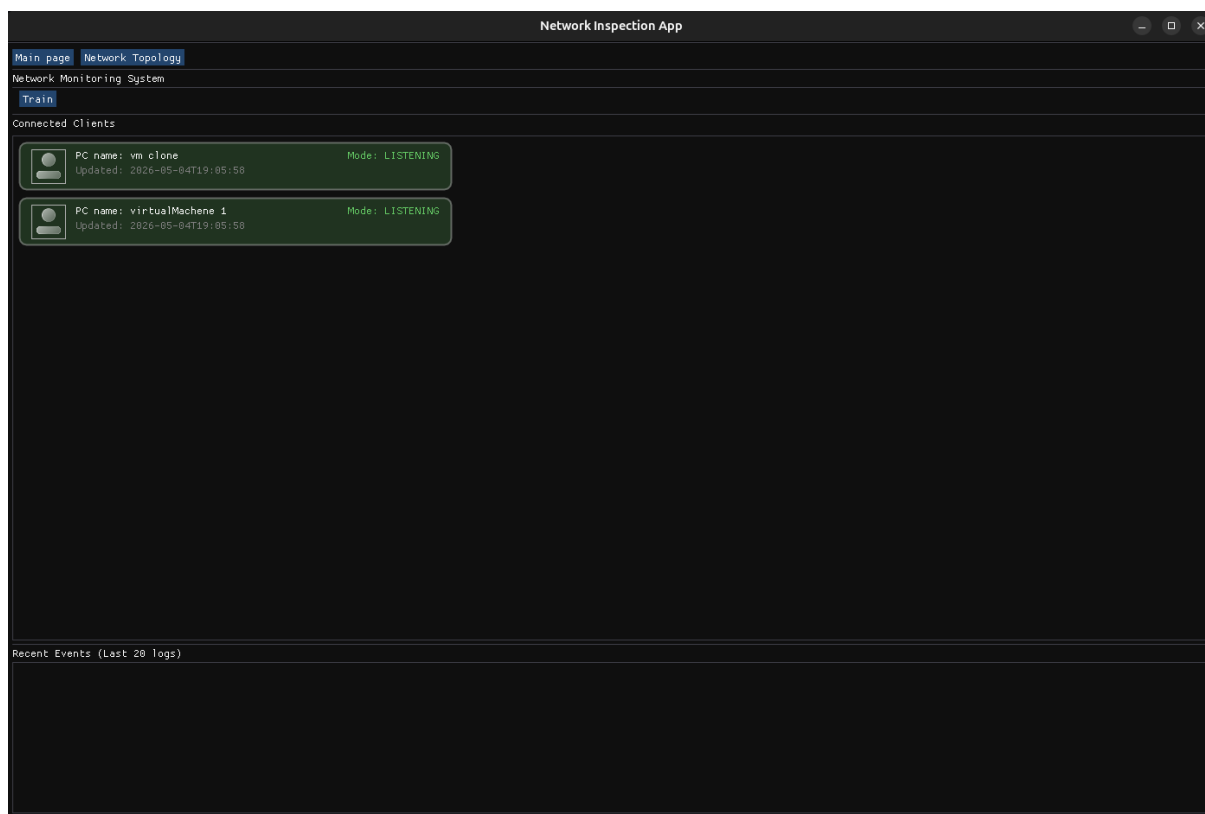


Figura 10 - Pagina principală

Sub lista de echipamente se regăsește secțiunea „*Recent Events (Last 20 logs)*”, ce reprezintă o listă derulabilă separată, cu cele mai recente 20 de evenimente raportate de oricare dintre echipamente. Aceste evenimente sunt separate printr-o linie orizontală. Fiecare linie este colorată în funcție de modul de funcționare activ la momentul raportării și are forma:

[oră] adresă_sursă:port_sursă -> adresă_destinație:port_destinație [PROTOCOL],

unde protocolul este afișat prin numele acestuia (TCP, UDP, ICMP etc.).

Pagina individuală a echipamentului se deschide cu o secțiune de antet: în stânga, o imagine de 80x80 de pixeli reprezentând iconița echipamentului, iar în dreapta acesteia, identificatorul unic al echipamentului și data ultimei actualizări [Figura 11 - Pagina echipamentului]. Sub aceasta se află un câmp de text editabil *nume*: prin care administratorul poate atribui un nume echipamentului, și un câmp *File Path*: însoțit de butonul *Load Image*. Adăugarea căii unei imagini și apăsarea butonului va încerca să încarce fișierul aflat la locația specificată și să îl afișeze într-o fereastră modală *Icon Preview*, ca și o imagine de 128x128 de pixeli. Această nouă fereastră va cuprinde de asemenea două butoane. Primul buton este *Use This Icon*, prin care, dacă imaginea a fost încărcată cu succes și se dorește setarea acesteia, va confirma intenția de modificare a iconiței cu cea afișată, schimbând și eliberând iconița anterioară și

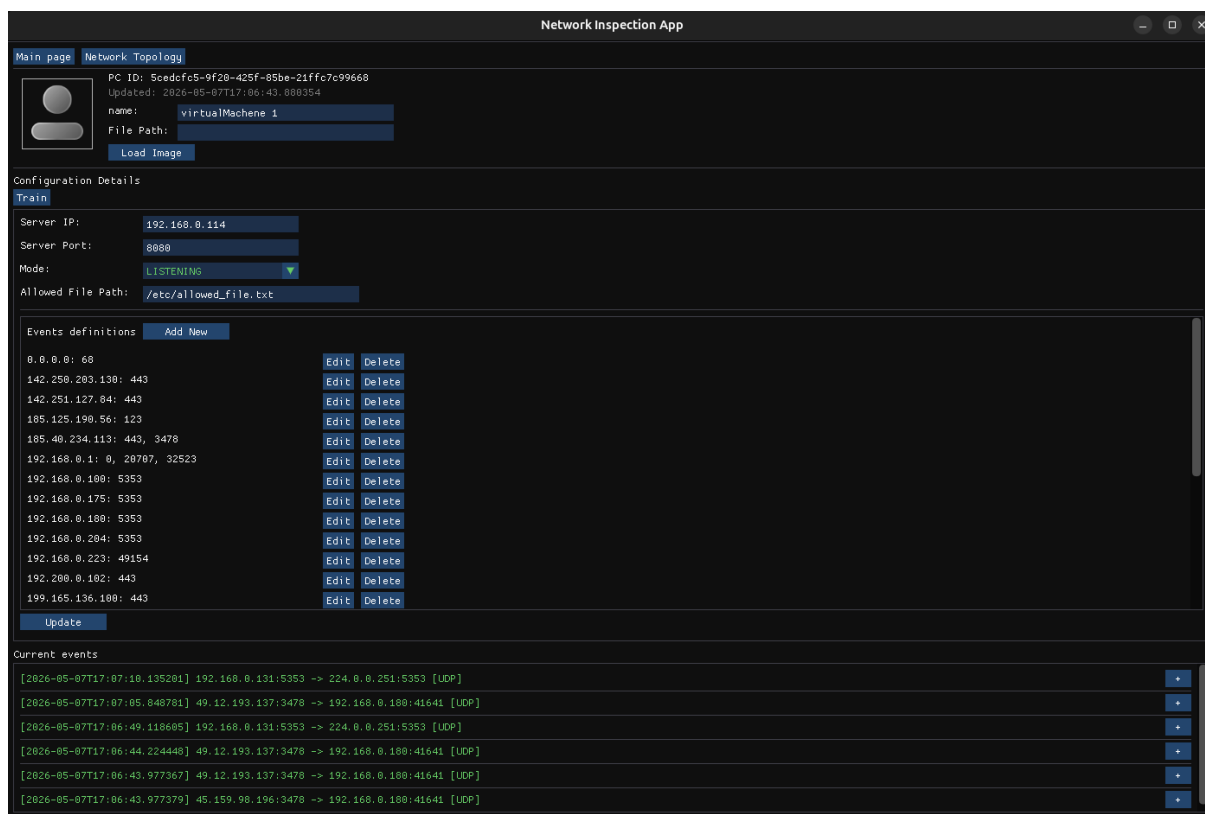


Figura 11 - Pagina echipamentului

setând imaginea nouă ca și iconița echipamentului. Cel de al doilea buton va cuprinde textul *Cancel*, ce va anula operația de actualizare.

Sub secțiunea de antet se află titlul *Configuration Details*, urmat de butonul „Train”/„Stop Training”, cu funcționalitate identică celui din pagina principală, dar limitat la echipamentul curent. Atunci când există modificări nesalvate în pagină – fie în configurație, fie în lista de conexiuni permise, fie în câmpurile generale – va fi afișată în continuarea acestui buton un al doilea buton intitulat *Discard changes*, care anulează toate modificările locale și reîncarcă datele primite din partea serverului, iar în colțul din dreapta al secțiunii apare textul *UNSAVED CHANGES*, scris cu roșu, ce avertizează vizual administratorul că starea afișată este diferită de cea de pe server, aplicația având modificări nesalvate.

Secțiunea de configurare propriu-zisă conține următoarele câmpuri modificabile, dispuse vertical:

- *Server IP*: un câmp de tip text pentru adresa componentei centrale
- *Server Port*: câmp numeric pentru portul de comunicare; acestui câmp îi este verificată validitatea, iar introducerea unei valori în afara intervalului 1-65535 va declanșa o

fereastră modală de avertizare, „*Invalid Port*”, cu textul „*Port must be between 1 and 65535!*” și un buton „*OK*”, ce va închide fereastra.

- *Mode*: o componentă de tip *dropdown* ce listează modurile de funcționare disponibile: *LISTENING*, *MONITORING*, *REACTIVE*, *DISABLED*. Acestea sunt afișate fiecare cu propria culoare, identică celei folosite pentru codificarea cardurilor din pagina principală
- *Allowed File Path*: ce va conține un câmp de tip textual ce are calea fișierului de conexiuni permise asociat echipamentului.

Sub aceste câmpuri se află secțiunea *Event definitions*, care conține lista editabilă a conexiunilor permise, precedată de butonul *Add New*. Fiecare element din listă va avea în continuare acestora două butoane: *Edit* și *Delete*. Prin apăsarea butonului de *Delete* se va șterge din listă elementul asociat butonului, iar prin apăsarea butonului *Edit* sau *Add New*, se va deschide o fereastră modală *Edit IP Entry*, ce conține un câmp de tip text, un indicator textual al formatului acceptat „*127.0.0.1 or 127.0.0.1:8080,8081*”, și două butoane, *Save* și *Cancel*, ce vor oferi posibilitatea de anulare a intenției de adăugate sau a salvării modificărilor acestora. În cazul butonului *Edit*, acest câmp va fi prepopulat cu informațiile liniei asociate butonului.

La finalul secțiunii de configurare se află butonul *Update*, care transmite toate modificările acumulate – configurația, lista de conexiuni permise, numele și iconița – către componenta centrală.

Sub secțiunea de configurare se află secțiunea *Current events* ce afișează, similar paginii principale, istoricul evenimentelor recente, dar informațiile sunt filtrate la cele raportate de echipamentul curent. Particularitatea vizuală acestei liste o reprezintă prezenta, pe fiecare linie ce reprezintă un eveniment, a unui buton, de 30x20 de pixeli, marcat cu simbolul de plus „+”. Apăsarea acestui buton adaugă automat adresa sursă a evenimentului respectiv în lista de conexiuni permise. În cazul în care dispozitivul ce a declanșat evenimentul se află deja în lista de conexiuni, dar nu a folosit unul dintre porturile permise de comunicare, portul folosit în evenimentul raportat este adăugat pe linia reprezentativă a acestuia, iar în cazul în care dispozitivul nu se afla în listă, acesta este adăugat alături de portul folosit.

Pagina Topologiei Rețelei [Figura 12 - Pagină topologie] prezintă un canvas cu fundal gri-antracit, prezent sub bara de navigare.

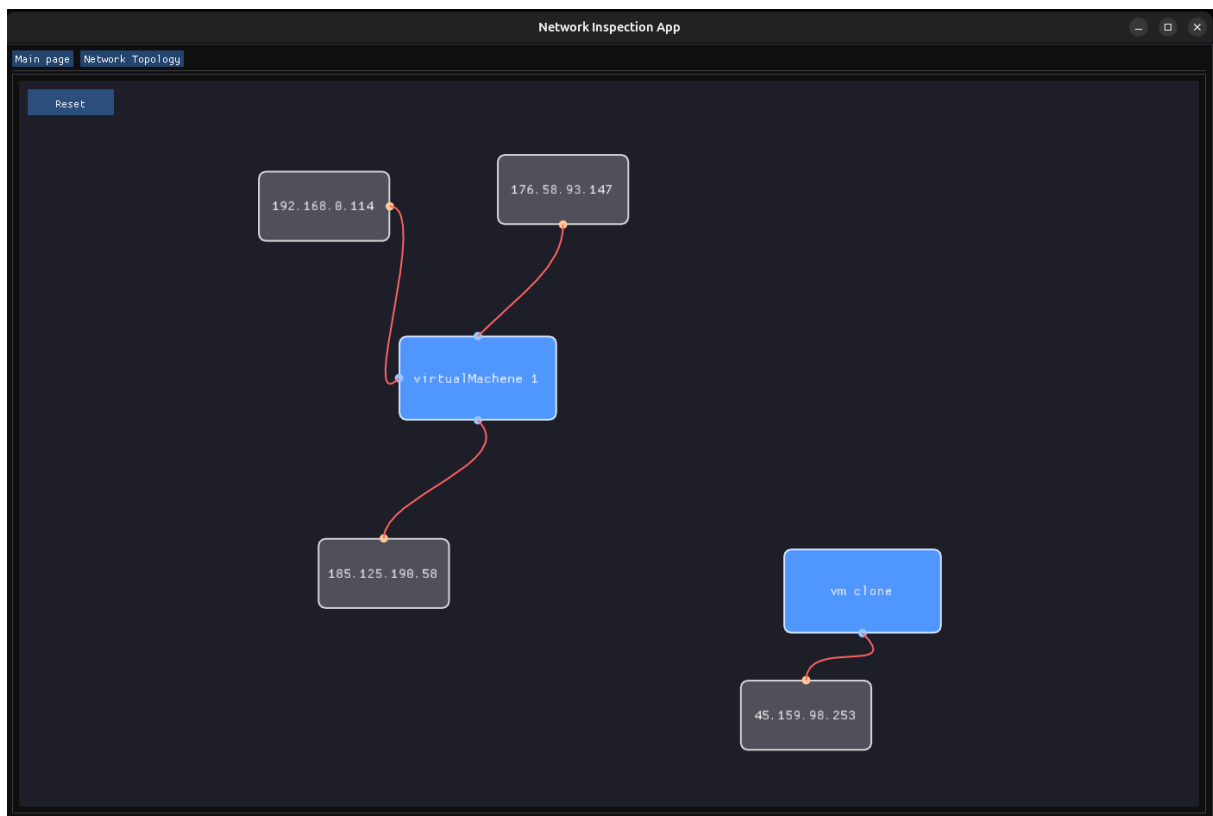


Figura 12 - Pagină topologie

Echipamentele monitorizate sunt reprezentate prin noduri de culoare albastră, de dimensiune ușor mai mare decât nodurile corespunzătoare dispozitivelor externe, care reprezintă elementele raportate, care sunt de culoare gri. Amândouă tipurile de noduri au un contur alb și colțuri rotunjite, cu numele echipamentului sau cu adresa IP centrate în interior. Conexiunile dintre noduri sunt desenate sub formă de curbe colorate reprezentativ: verde pentru conexiunile provenite de la o adresă externă și permise conform configurației și roșu, pentru o astfel de conexiunile care nu se regăsește în lista de conexiuni permise a echipamentului destinatar.

Trecerea cursorului peste un nod îl evidențiază printr-un contur galben și afișează o fereastră de tip *tooltip*. Pentru un echipament monitorizat, informația afișată va cuprinde indicatorul acestuia, numele, modul curent de funcționare și, separat printr-o linie, lista tuturor adreselor IP folosite de echipament. În cazul unui dispozitiv extern, dacă conexiunea a fost identificată și blocată, informația va fi afișată cu fundal roșu, iar în cazul în care comunicarea a fost doar raportată, aceasta va avea fundal galben. Informațiile afișate pentru elementele externe cuprind porturile sursă și destinație observate, adresele IP, protocolul și adresele MAC asociate.

Un click dreapta pe un nod ce reprezintă un echipament monitorizat deschide un meniu contextual cu o singură opțiune, *Edit Config*, care face navigarea către pagina individuală a echipamentului respectiv. Vizualizarea suportă funcționalitatea de apropiere și îndepărtare prin roțița *mouse*-ului, cu factor de scalare între 0.3x și 3x, ce păstrează fix punctul aflat sub cursor, și de mutare în imagine prin tragere cu butonul stâng al *mouse*-ului apăsat, permițând explorarea liberă a topologiei rețelei, indiferent de numărul de echipamente afișate.

În colțul din stânga-sus al paginii se află butonul *Reset*, de dimensiune 100x30 de pixeli, care, prin apăsare, este responsabil de resetarea datelor de topologie acumulate de pe server, ce are ca și consecință ștergerea tuturor dispozitivelor neurmărite și recalcularea de la zero a pozițiilor nodurilor ce reprezintă echipamentele. Această opțiune devine utilă când reprezentarea grafică cuprinde un număr prea mare de informații care au devenit irelevante.

Concluzii și planuri de viitor

Conceptul aplicației prezentate reprezintă o soluție funcțională de securizare a echipamentelor în medii industriale atunci când comunicarea dintre componentele sistemului este bine conturată, iar elementele ce au drepturi de comunicare externă pot fi definite și trebuie controlate strict. Aplicația se bazează pe un mecanism simplificat de control al accesului pe bază de politici, prin definirea explicită a conexiunilor permise între echipamente.

Prin monitorizarea traficului în rețea și aplicarea regulilor de configurare, sistemul permite limitarea comunicării neautorizate și oferă control asupra interacțiunilor dintre componentele infrastructurii. Abordarea utilizată facilitează identificarea rapidă a conexiunilor noi sau neprevăzute și reduce suprafața de atac a dispozitivelor monitorizate.

În cadrul testelor efectuate asupra performanței sistemului [Tabelul 4 - durata medie de procesare], a fost măsurat timpul de procesare al pachetelor la nivelul componentei de filtrare. Valorile prezentate în tabel reprezintă durata medie (*mean value*), exprimată în nanosecunde (*ns*), necesară unui pachet să fie identificat ca și permis sau rejectat, să i se extragă informațiile și să iasă din componentă. Durata de rulare a fiecărui set de date este de aproximativ 5 minute. Pentru testare s-a folosit un procesor *Ryzen 5 7600* cu o frecvență de bază de 3800MHz și o conexiune de 1Gbps. Aceste rezultate oferă o estimare a *overhead*-ului introdus de sistem asupra dispozitivelor monitorizate și oferă o bază pentru analiza funcționării soluției în condiții de trafic.

Durata (<i>ns</i>)	Număr de pachete procesare
983	42
1403	498
1552	306
1703	293
1633	321

Tabelul 4 - durata medie de procesare

Pentru o verificare mai detaliată a capacităților aplicației se poate face o analiză suplimentară realizată asupra *overhead*-ului pe dispozitive reprezentative mediului industrial. Rezultatele pot fi puse în comparație cu performanța sistemului fără prezența aplicației, pentru a avea o imagine clară asupra consecințelor introducerii mecanismului de filtrare la nivel *kernel*. Totodată, în această testare poate apărea ca variabilă numărul de elemente din lista conexiunilor permise, pentru a evidenția impactul acestuia asupra timpului de procesare și a scalabilității soluției în condiții de trafic variabil.

În concluzie, aplicația reprezintă o bază funcțională a unei aplicații de filtrare, vizată mediului industrial, ce poate reprezenta baza unor soluții mai avansate de securitate. Aceasta se concentrează pe un echilibru între control, simplitate și flexibilitatea de extindere ulterioară.

În cadrul dezvoltărilor viitoare, aplicația poate fi extinsă prin adăugarea unor mecanisme de învățare a tiparelor considerate normale în rețea și identificarea deviațiilor de la aceste tipare. Reacționarea automată pentru un număr de cazuri poate reprezenta de asemenea o funcționalitate utilă aplicației: în cazul unui dispozitiv configurat să accepte comunicații exclusiv de la echipamente definite explicit — mod de funcționare REACTIVE — detectarea unui număr anormal de pachete provenite de la o sursă acceptată poate indica o activitate suspectă. Aplicația poate reacționa prin notificări către administrator sau blocarea emițătorului, în funcție de configurare.

Bibliografie

- [1] W. Bolton, 2015, *Programmable Logic Controllers*, ediția a șasea, Amsterdam, Olanda; Elsevier/Newnes, ISBN: 978-0-12-802929-9.
- [2] G. Yadav și K. Paul, 2021, „Architecture and security of SCADA systems: A review,” *International Journal of Critical Infrastructure Protection*, vol. 34, Art. no. 100433, doi: 10.1016/j.ijcip.2021.100433.
- [3] G. Thomas, 2008, „Introduction to the Modbus Protocol”, *The Extension, A Technical Supplement to Control Network*, vol 9, issue 4
- [4] D. Beresford, 2011 „Exploiting Siemens Simatic S7 PLCs”, conferința *Black Hat USA - 2011*
- [5] G. Clarke, D. Reynders, E. Wright, 2004, „Practical Modern SCADA Protocols: DNP3, 60870.5 and Related Systems”, Elsevier/Newnes, ISBN: 07506-6799-5
- [6] S. T. Bushby, și H. M. Newman, 2002, „Significant New Features and Future Enhancements,” *BACnet Today: A Supplement to ASHRAE Journal*, ASHRAE Journal, p 10-16
- [7] P. Brooks, 2001, „Ethernet/IP-industrial protocol” ETFA 2001. 8th International Conference on Emerging Technologies and Factory Automation. Proceedings, Antibes-Juan les Pins, France, pp. 505-514 vol.2, doi: 10.1109/ETFA.2001.997725.
- [8] A. Moise, J. Brodtkin, 2011, „ANSI C12.22, IEEE 1703, and MC12.22 Transport Over IP”, *Internet Engineering Task Force (IETF)*, RFC 6142, ISSN: 2070-1721
- [9] M. Nawrocki, T. C. Schmidt și M. Wählisch, 2020, „Uncovering Vulnerable Industrial Control Systems from the Internet Core”, arXiv preprint arXiv:1901.04411.
- [10] I. Lesnic, 2026, „Analiza tipuri de atacuri cibernetice și a metodelor de protecție” Teză de master, *Universitatea Tehnică a Moldovei, Chișinău, Republica Moldova*
- [11] A. Cui, M. Costello, S. Stolfo, 2013, „When firmware modifications attack: A case study of embedded exploitation”, doi: 10.7916/D8P55NKB.
- [12] C. Smiliotopoulos, G Kambourakis, C Kilias, 2024, „Detecting lateral movement: A systematic survey”, *Heliyon*, 10 e26317
- [13] J. F. Miller, 2013 „Supply Chain Attack Framework and Attack Patterns” *MITRE Technical Report MTR140021*, Project No. 0713D050-AA, Contract No. W15P7T-13-C-F600, *The MITRE Corporation*
- [14] B. Meixell and E. Forner, 2013, „Out of control: Demonstrating scada exploitation,” *Black Hat*, p. 2013.
- [15] J. Klick, S. Lau, D. Marzin et al., 2015, „Internet-facing PLCs – A new back orifice,” *Blackhat USA*.
- [16] Baezner, Marie; Robin, Patrice, 2017, „Hotspot Analysis: Stuxnet, October 2017”, Center for Security Studies (CSS), ETH Zürich.

- [17] R. M. Lee, M. J. Assante, și T. Conway, Mar. 2016, „*Analysis of the Cyber Attack on the Ukrainian Power Grid – Defense Use Case*”, Electricity Information Sharing and Analysis Center (E-ISAC) / SANS ICS, Washington, DC, USA.
- [18] J. S. White, T. T. Fitzsimmons și J. N. Matthews, 2013, „*Quantitative Analysis of Intrusion Detection Systems: Snort and Suricata*”, Clarkson University, Potsdam, NY, Proc. SPIE 8757, Cyber Sensing 2013, 875704
- [19] T. K. Das, S. Adepu și J. Zhou, 2020, „*Anomaly detection in Industrial Control Systems using Logical Analysis of Data*”, Computers & Security, vol. 96, art. 101935.
- [20] J. Kim, J.-H. Yun și H. C. Kim, 2019, „*Anomaly Detection for Industrial Control Systems Using Sequence-to-Sequence Neural Networks*”, arXiv preprint arXiv:1911.04831.
- [21] M. Elnour, M. Noorizadeh, M. Shakerpour, N. Meskin, K. Khan și R. Jain, 2023, „*A Machine Learning Based Framework for Real-Time Detection and Mitigation of Sensor False Data Injection Cyber-Physical Attacks in Industrial Control Systems*”, vol. 11, pp. 86977–86998, doi: 10.1109/ACCESS.2023.3303015.
- [22] L. Wang, 2026, „*Clustering-Enhanced Domain Adaptation for Cross-Domain Intrusion Detection in Industrial Control Systems*”, arXiv preprint arXiv:2604.12183.
- [23] Documentație Netfilter, disponibil la: <https://www.netfilter.org/> accesat la: 17 mai 2026
- [24] Documentație Graphviz, disponibil la: <https://graphviz.org/> accesat la: 17 mai 2026.
- [25] Adeeb Alhomoud, R. Munir, J. P. Disso, I. Awan și A. Al-Dhelaan, 2011, „*Performance Evaluation Study of Intrusion Detection Systems*”, Procedia Computer Science, vol. 5, pp. 173–180.
- [26] The Linux Documentation Project, „Guides”, disponibil la: <https://tldp.org/guides.html> accesat la: 15 mai 2026.
- [27] Raik Niemann, Udo Pfingst, Richard Göbel, 2015, „Performance Evaluation of netfilter: A Study on the Performance Loss When Using netfilter as a Firewall”, arXiv preprint arXiv:1502.05487
- [28] Documentație SQLite, disponibil la: <https://sqlite.org/docs.html> accesat la 1 mai 2026
- [29] S. Fritsch, M. Simon și S. Gallenmüller, 2025, „*Firewalling with eBPF: A Performance Comparison of XDP-based Solutions*”, Seminar IITM SS 25, Technical University of Munich, doi: 10.2313/NET-2025-11-3_04.
-